

UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Faculty of Engineering, Built Environment and Information Technology

EBB 320

Control Systems

Practical 1: Hardware Implementation and System Identification

Name and Surname	Student Number	Signature	% Contribution
Hanré Coetzer	21507334		33
Cornelis Claasen	24704611		33
B.J. Mncube	23700816		33

By signing this assignment we confirm that we have read and are aware of the University of Pretoria's policy on academic dishonesty and plagiarism and we declare that the work submitted in this assignment is our own as delimited by the mentioned policies. We explicitly declare that no parts of this assignment have been copied from current or previous students' work or any other sources (including the internet), whether copyrighted or not. We understand that we will be subjected to disciplinary actions should it be found that the work we submit here does not comply with the said policies.

October 2, 2026

Contents

1 Theoretical Background **2**

2 Design and Simulation **3**
2.1 P Controller 4
2.2 PI Controller 8
2.3 PD Controller 11
2.4 PID - Reference Tracking Controller 14
2.5 PID - Disturbance Rejection Controller 16

3 Implementation **19**
3.1 PID Control on MCU 19
3.1.1 Discrete PID Algorithm and Hardware Protection 19

4 Results **21**
4.1 PID Reference Tracking 21
4.2 PID Disturbance Rejection 23

5 Discussion **25**
5.1 Proportional Term 25
5.2 Integral Term 25
5.3 Derivative Term 25
5.4 Controller Comparison 26

6 Code **27**
6.1 Matlab Code 27
6.2 Model Fit Code 41
6.3 Prac 2 — Controller Design (P and PI) 44
6.4 Prac 2 — Report Figure Generation (P and PI) 44
6.5 MCU Code 44

1 Theoretical Background

The accuracy of the estimated transfer function models was evaluated using the Normalized Root Mean Square Error (NRMSE) fit percentage. This metric quantifies how well the simulated model response matches the measured physical data, and is defined as:

$$\text{Fit } (\%) = 100 \times \left(1 - \frac{\|y - \hat{y}\|}{\|y - \bar{y}\|}\right) \tag{1}$$

where y is the measured output data, $\hat{y}$ is the simulated model output, $\bar{y}$ is the mean of the measured output data, and $\|\cdot\|$ denotes the Euclidean norm.

2 Design and Simulation

The previous First Order Plus Dead Time (FOPDT) models made using Matlab's `tfest` function were used to construct the simulation models used in this practical.

$$G_{11} = \frac{0.1175}{s + 0.007592} \tag{2}$$

$$G_{12} = \frac{0.05949}{s + 0.0060685} \tag{3}$$

Matlab's `compare` function was then used to confirm G_{11}'s accuracy. It was found to be $\approx 94\%$ accurate.

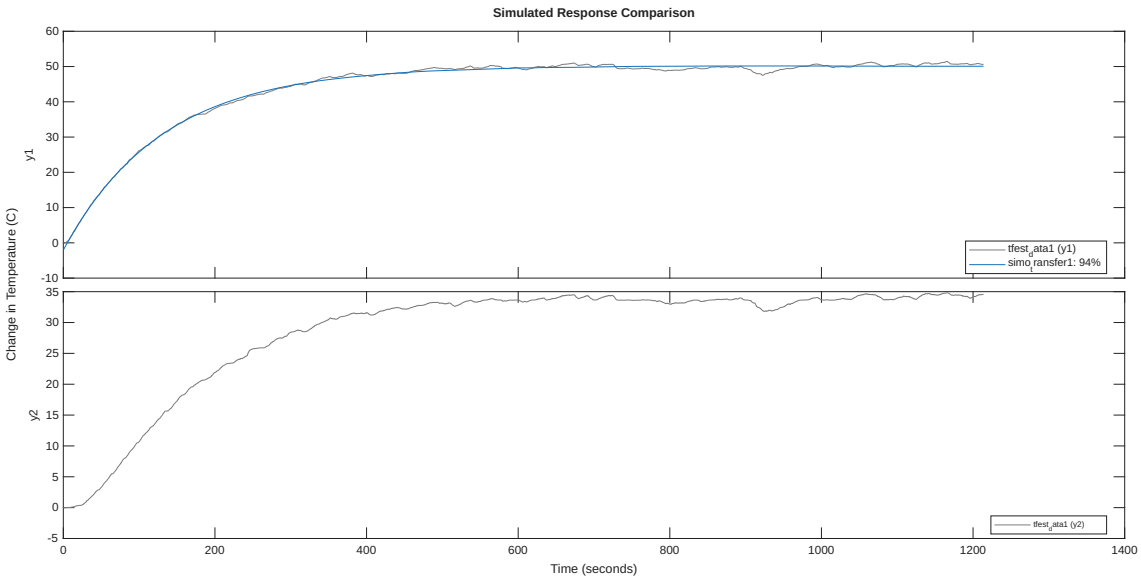


Figure 1: Accuracy comparison of G_{11}'s transfer function

2.1 P Controller

The Proportional (P) controller was designed using MATLAB's `pidtune` function applied to the transfer function G_{11} identified in Practical 1. The resulting controller has the form $C(s) = K_p$, where the proportional gain was tuned to achieve a phase margin of 60°. The P controller produces a fast initial response but introduces a steady-state error because, without an integral term, the controller output falls to zero as the error approaches zero.

Listing 1: P Controller Design (MATLAB)

```
[C1_p, info] = pidtune(G11, 'P');
```

The controller was then simulated using the Simulink model shown in figs. 2 and 3.

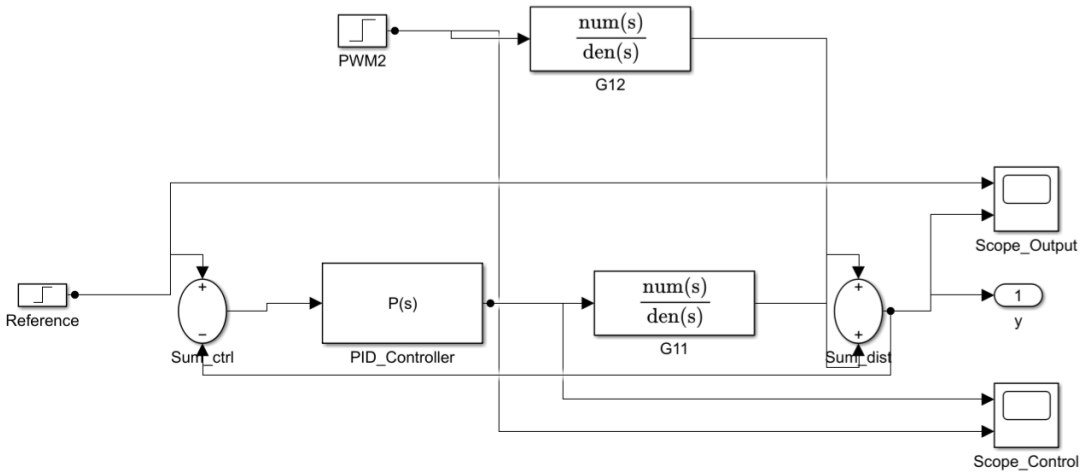


Figure 2: Simulink model used for reference tracking with no disturbance (Figure 1 from the practical guide). The controller $C(s)$ drives the plant G_{11} in a unity negative feedback loop.

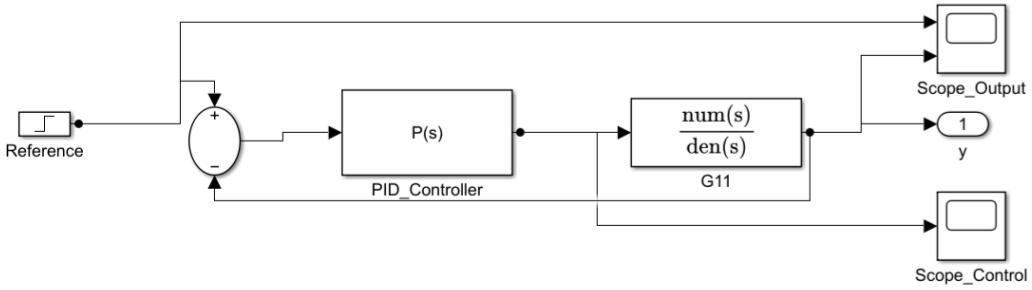


Figure 3: Simulink model used for reference tracking with disturbance (Figure 2 from the practical guide). The PWM2 input passes through G_{12} and is added to the plant output, simulating the thermal disturbance from transistor 2.

The tuned gain and step-response metrics for the closed-loop system $T(s) = \frac{C_P G_{11}}{1+C_P G_{11}}$ are summarised in tables 1 and 2.

Table 1: P Controller Tuned Parameters

Parameter	Value
K_p	0.847505
K_i	—
K_d	—
Achieved Phase Margin	60.0°

Table 2: P Controller Step Response Metrics (2% Settling Criterion)

Step Response Parameter	Closed Loop	Open Loop
Overshoot (%)	7.42	0.0
Rise Time (s)	7.9	291.2
Peak Time (s)	21.2	970.5
Settling Time (s)	30.3	524.4
Steady-State Gain	0.9296	1.0

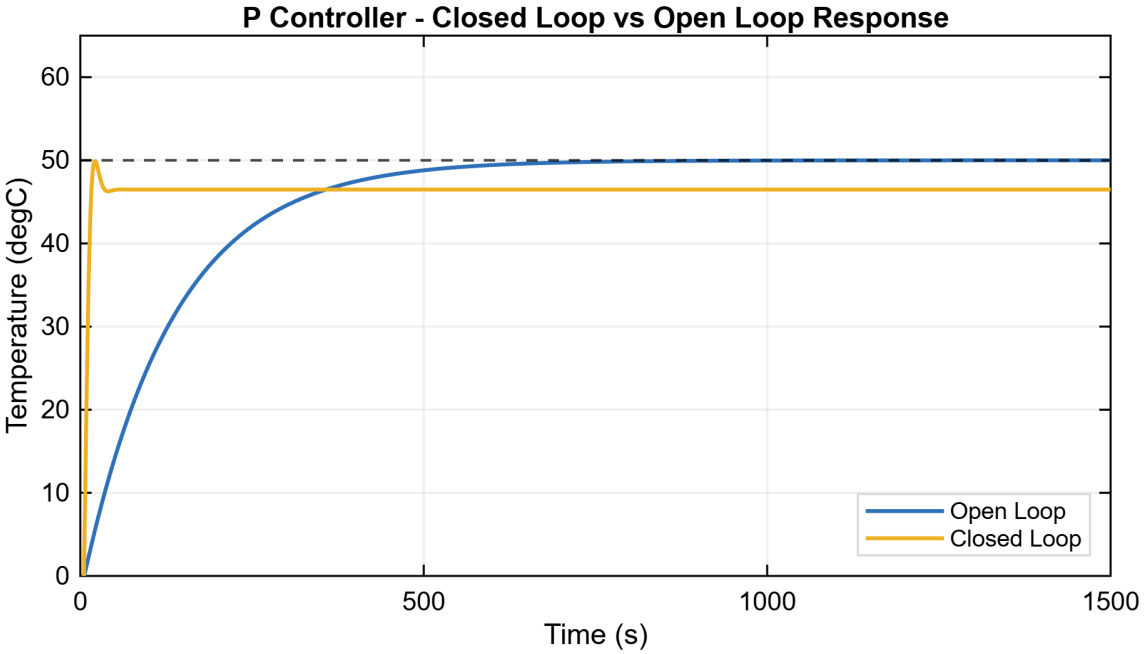


Figure 4: Closed-loop vs open-loop step response of the P controller. The closed-loop response reaches the reference significantly faster but exhibits overshoot and a steady-state error due to the absence of an integral term.

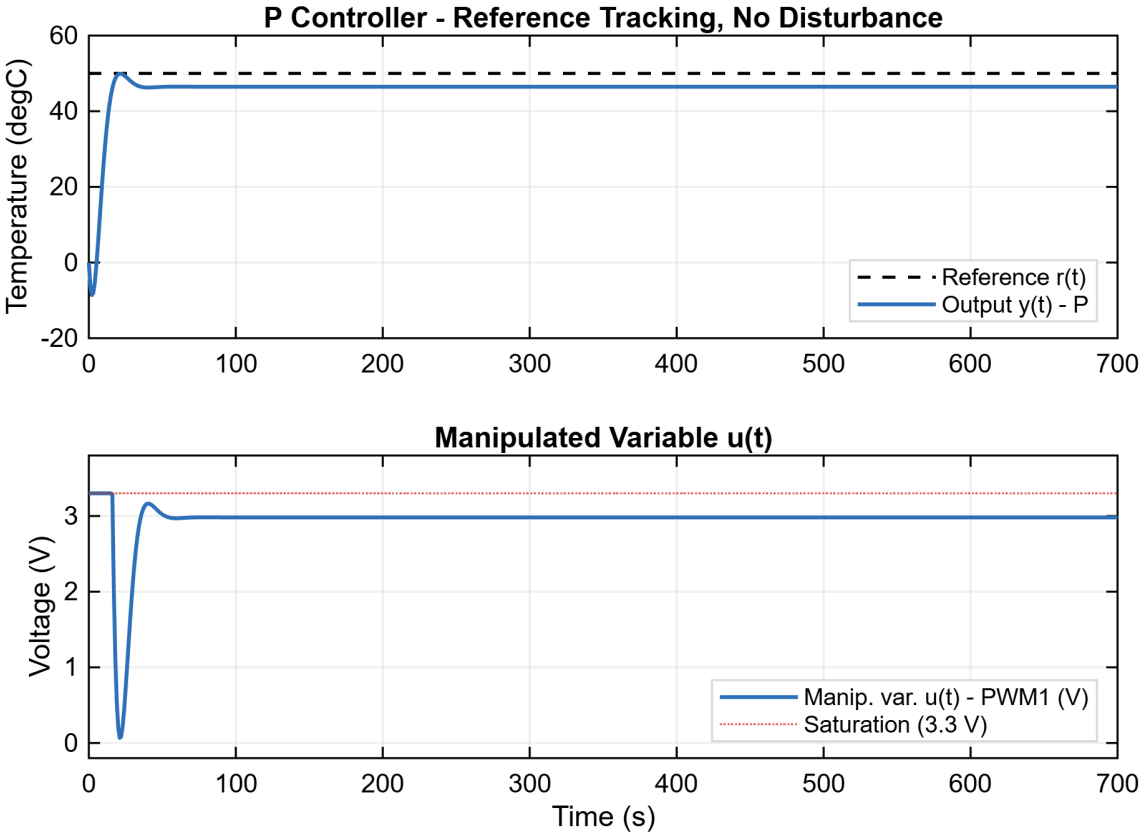


Figure 5: Closed-loop response of the P controller with no disturbance. Top: reference trajectory and controlled variable (temperature). Bottom: manipulated variable (PWM1 voltage).

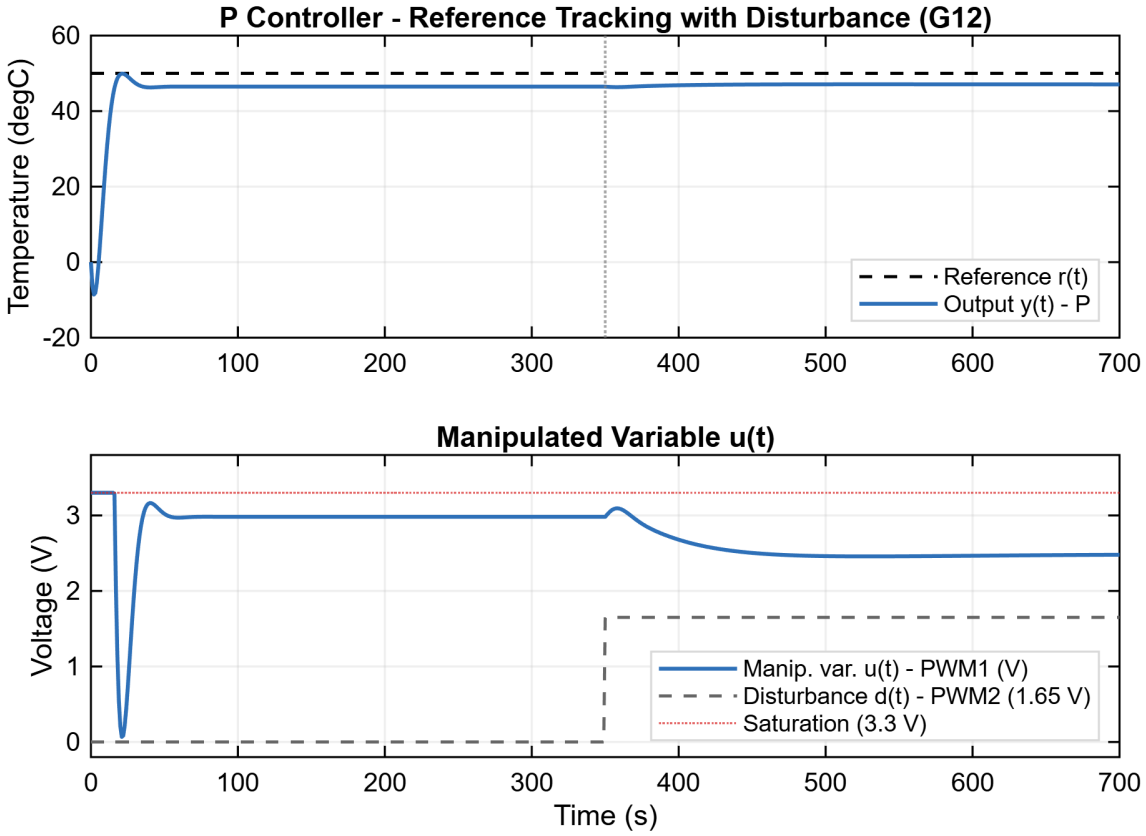


Figure 6: Closed-loop response of the P controller with disturbance via G_{12}. Top: reference and controlled variable. Bottom: manipulated variable (PWM1) and disturbance input (PWM2).

2.2 PI Controller

The Proportional-Integral (PI) controller was designed by adding an integral term to eliminate the steady-state error observed in the P controller. The controller has the form $C(s) = K_p + \frac{K_i}{s}$, which ensures that the integral term accumulates the error over time and drives the steady-state error to zero. The trade-off is a slower response and increased overshoot compared to the P controller. The same Simulink models shown in figs. 2 and 3 were used, with the PI gains loaded into the PID Controller block.

Listing 2: PI Controller Design (MATLAB)

```
[C1_pi, info] = pidtune(G11, 'PI');
```

The tuned gains and step-response metrics for the closed-loop system are summarised in tables 3 and 4.

Table 3: PI Controller Tuned Parameters

Parameter	Value
K_p	0.075342
K_i	0.001432
K_d	—
Achieved Phase Margin	60.0°

Table 4: PI Controller Step Response Metrics (2% Settling Criterion)

Step Response Parameter	Closed Loop	Open Loop
Overshoot (%)	12.95	0.0
Rise Time (s)	103.1	291.2
Peak Time (s)	232.6	970.5
Settling Time (s)	394.8	524.4
Steady-State Gain	1.0000	1.0

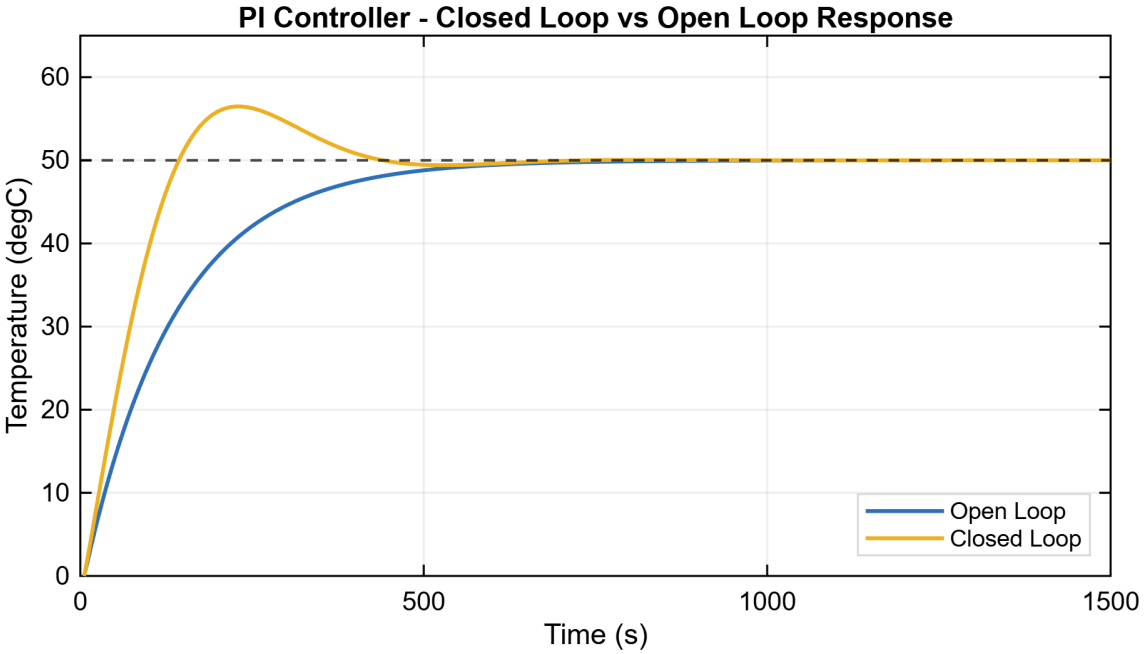


Figure 7: Closed-loop vs open-loop step response of the PI controller. The integral term eliminates the steady-state error present in the P controller, with the closed-loop output converging exactly to the 50 °C reference.

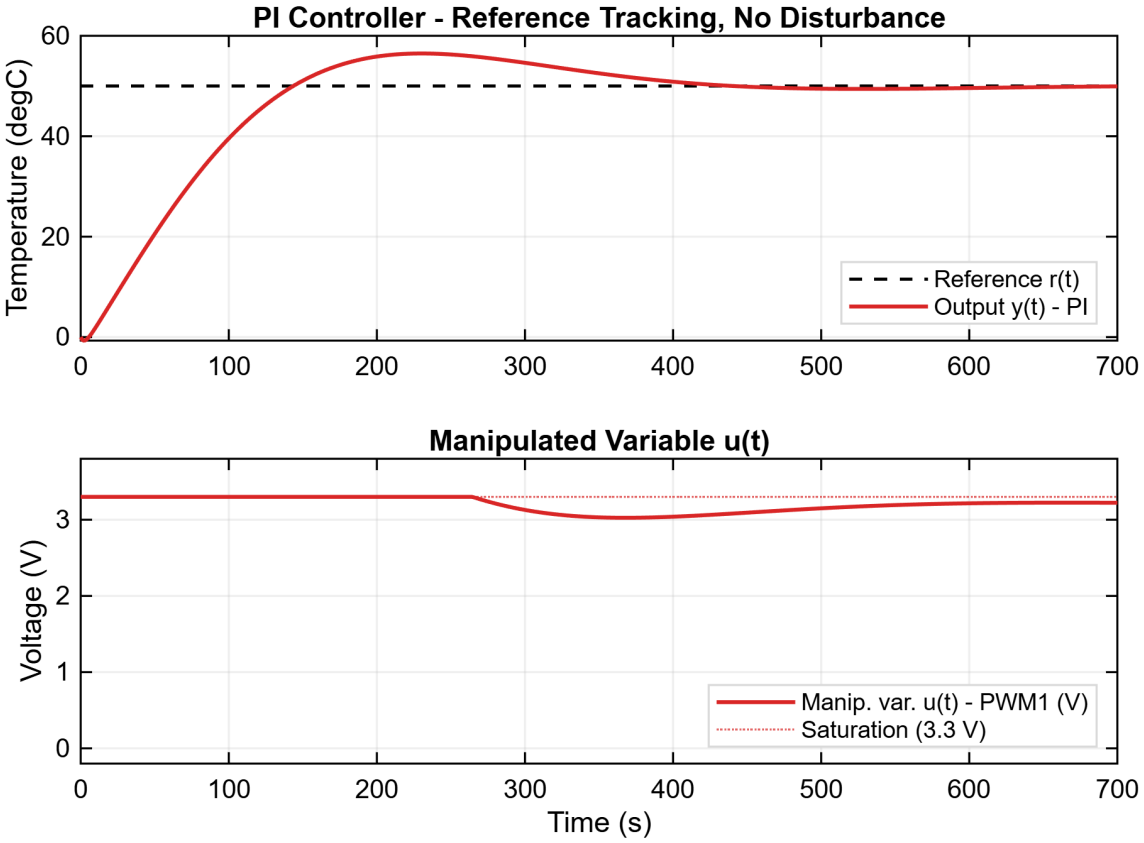


Figure 8: Closed-loop response of the PI controller with no disturbance. Top: reference trajectory and controlled variable (temperature). Bottom: manipulated variable (PWM1 voltage).

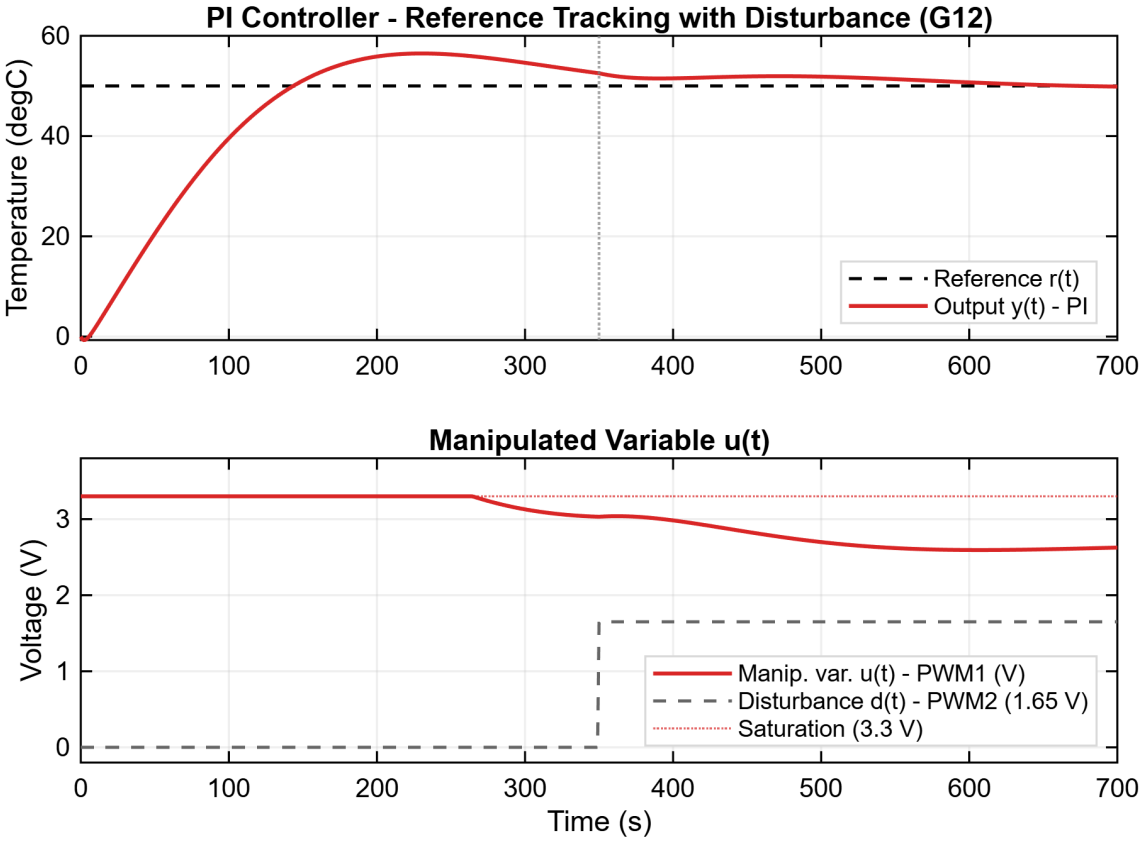


Figure 9: Closed-loop response of the PI controller with disturbance via G_{12}. Top: reference and controlled variable. Bottom: manipulated variable (PWM1) and disturbance input (PWM2).

2.3 PD Controller

The PID parameters were found using Matlab's `pidtune` function with G_{11} as the input. Those parameters were: $K_p = 4.58$, $K_i = 0.0$, and $K_d = 0.0$. The controller was then simulated using the Simulink model below. Where the step response will be shifted up, due to the addition of the ambient temperature.

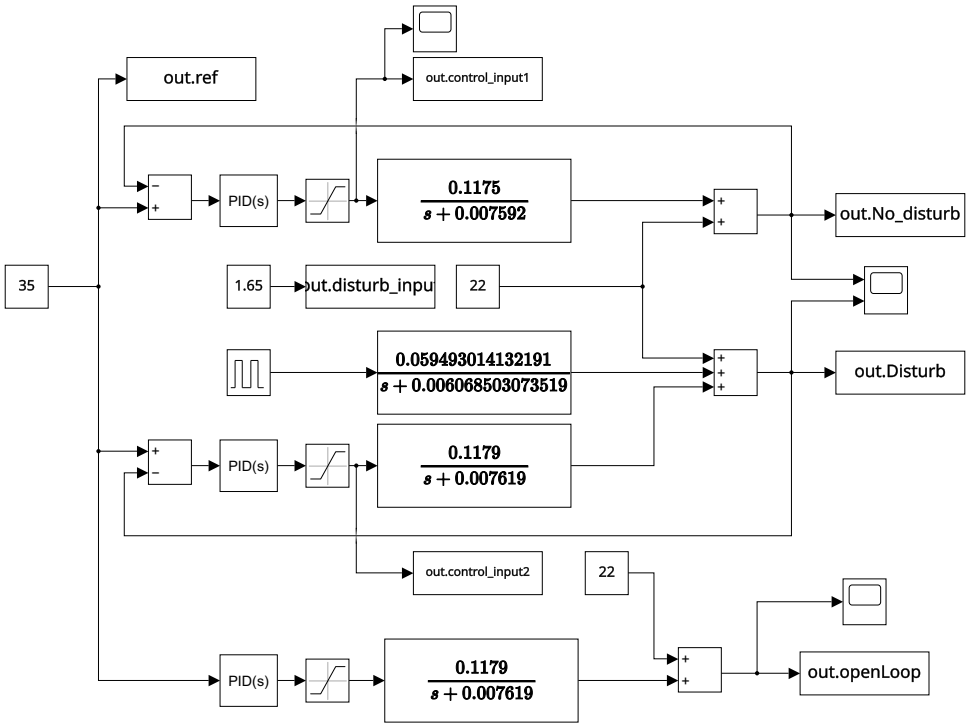


Figure 10: Simulink Model used for PD Controller

The resulting model was then used to find the following step response parameters using Matlab's `stepinfo` function.

Table 5: PD Controller Step Response Metrics

Step Response Parameter	Closed Loop	Open Loop
Overshoot (%)	8.8047×10^{-5}	0.0
Rise Time (s)	77.3698	291.2
Peak Time (s)	119.1159	970.5
Settling Time (s)	97.0706	524.4

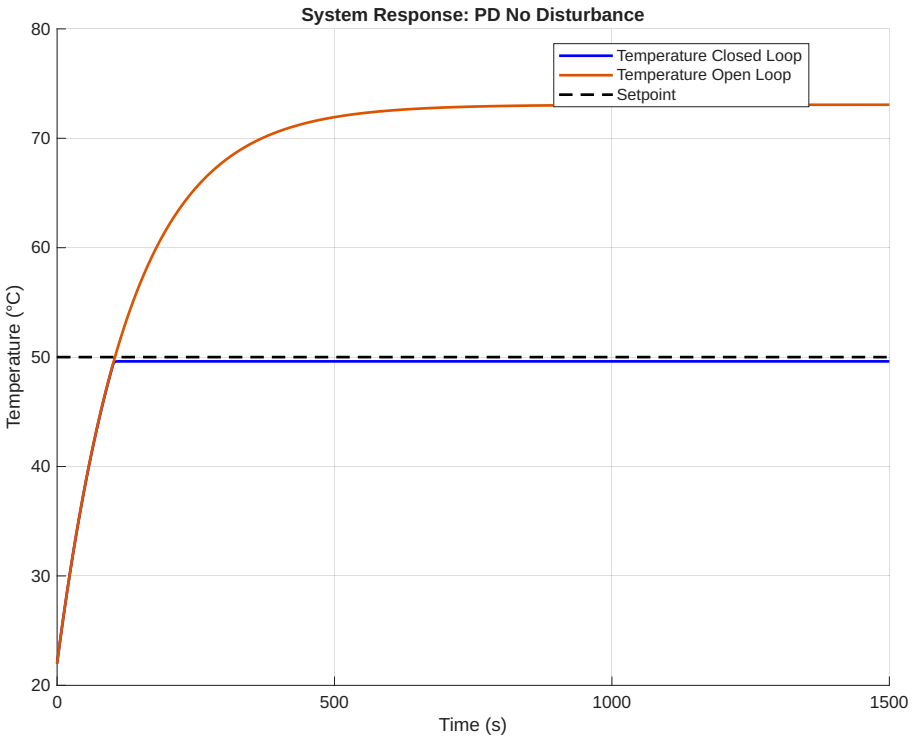


Figure 11: Comparing the closed loop PD controller step response to the open loop step response.

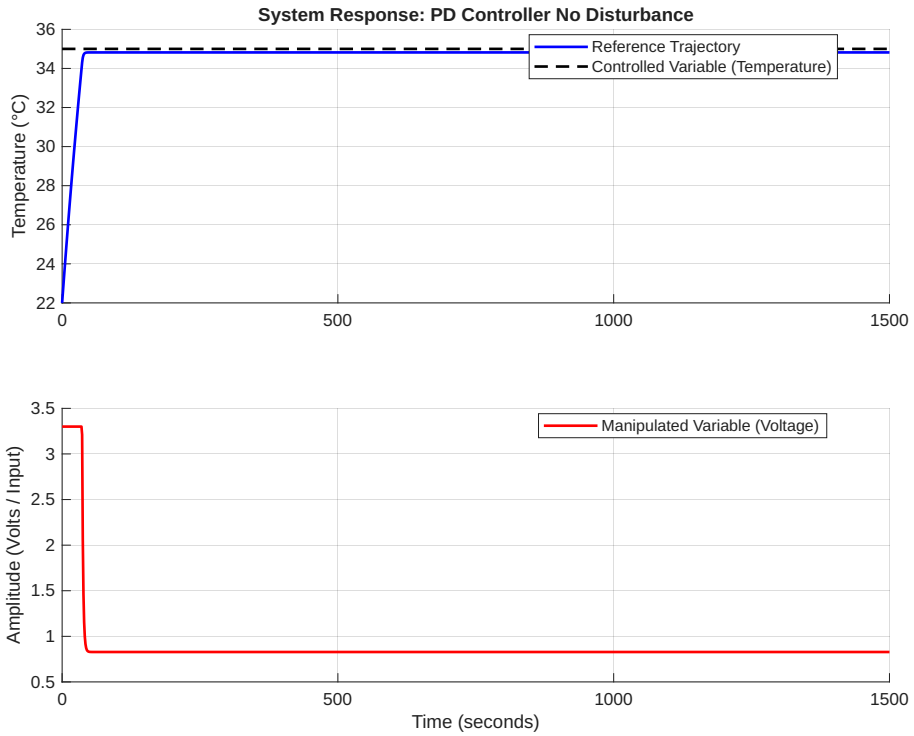


Figure 12: PD Control with no disturbance

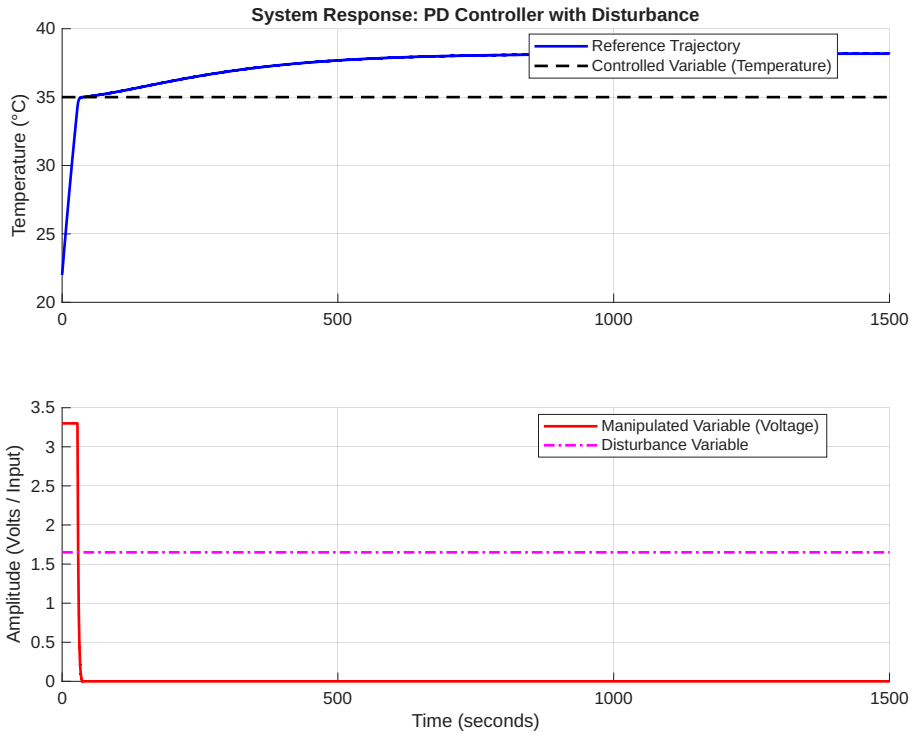


Figure 13: PD Control with disturbance

2.4 PID - Reference Tracking Controller

The parameters found using Matlab's `pidtune` function were: $K_p = 0.112$, $K_i = 0.000935$, and $K_d = 0.0$.

The same Simulink model used for the PD controller[10] was used for the PID reference tracking model.

The following step response parameters where then found using the model:

Table 6: Table comparing closed loop PID controller tuned for reference tracking against the open loop controller

Step Response Parameter	Closed Loop	Open Loop
Overshoot (%)	0.1592	0.0
Rise Time (s)	123.6591	291.2
Peak Time (s)	432.7289	970.5
Settling Time (s)	220.6166	524.4

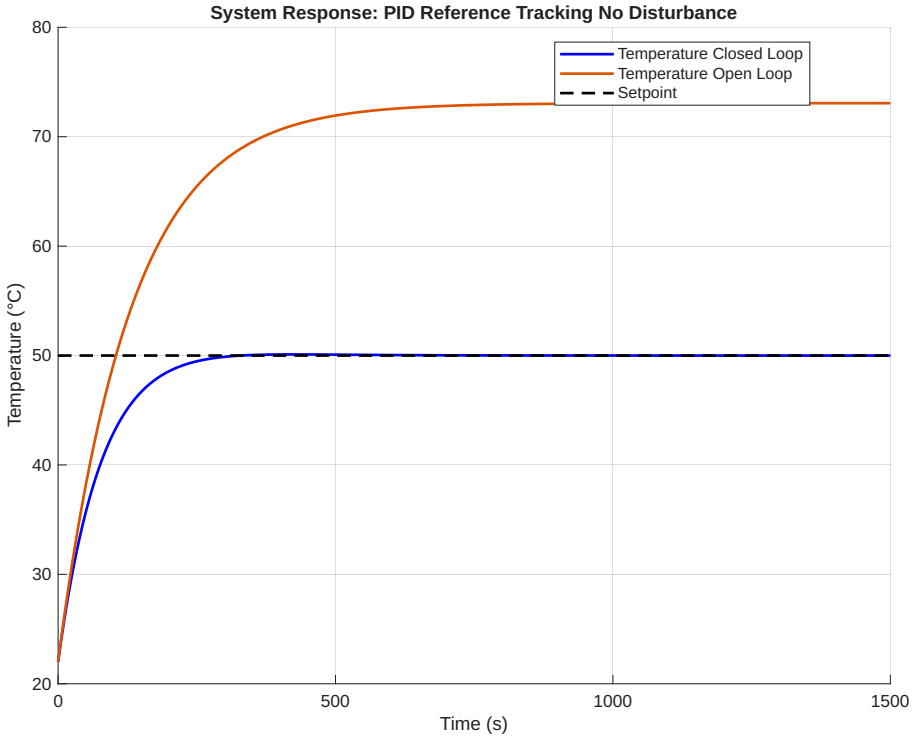


Figure 14: Comparing the closed loop step response for the PID reference tracking controller to the open loop step response

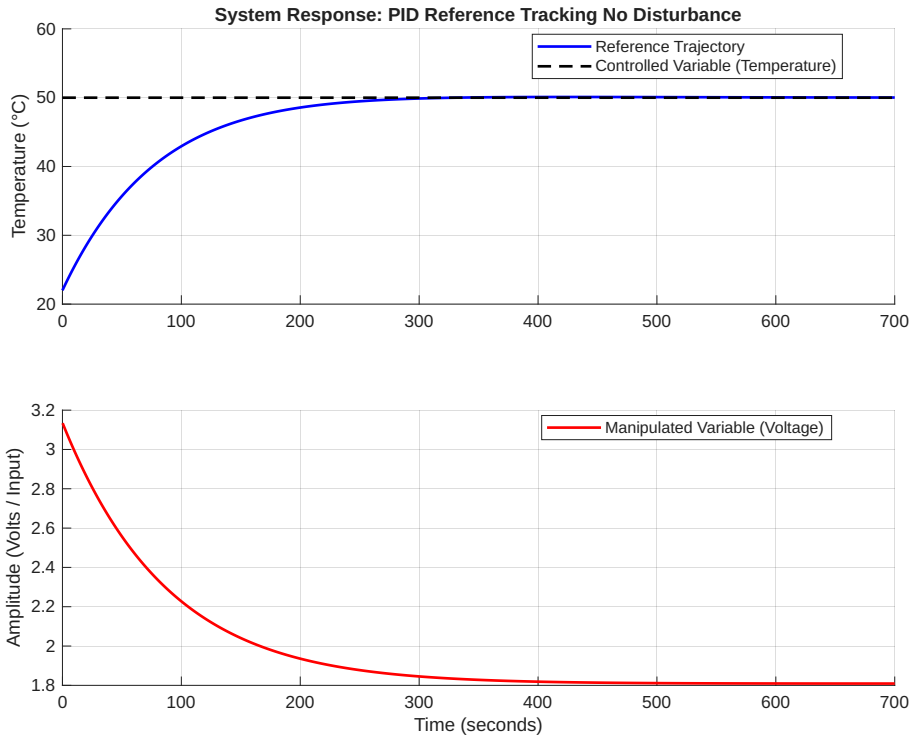


Figure 15: PID reference tracking control with no disturbance

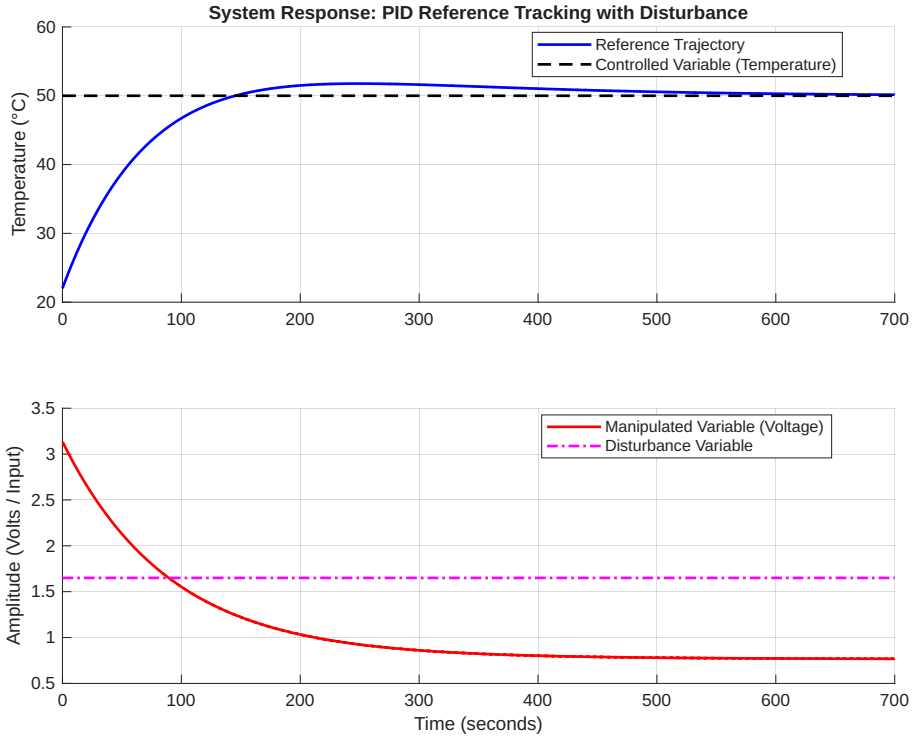


Figure 16: PID reference tracking control with disturbance

2.5 PID - Disturbance Rejection Controller

The parameters used for the disturbance rejection tuning are $K_p = 0.067424$, $K_i = 0.0016631$, and $K_d = 0.68337$.

The controller was then simulated using the Simulink Model below.

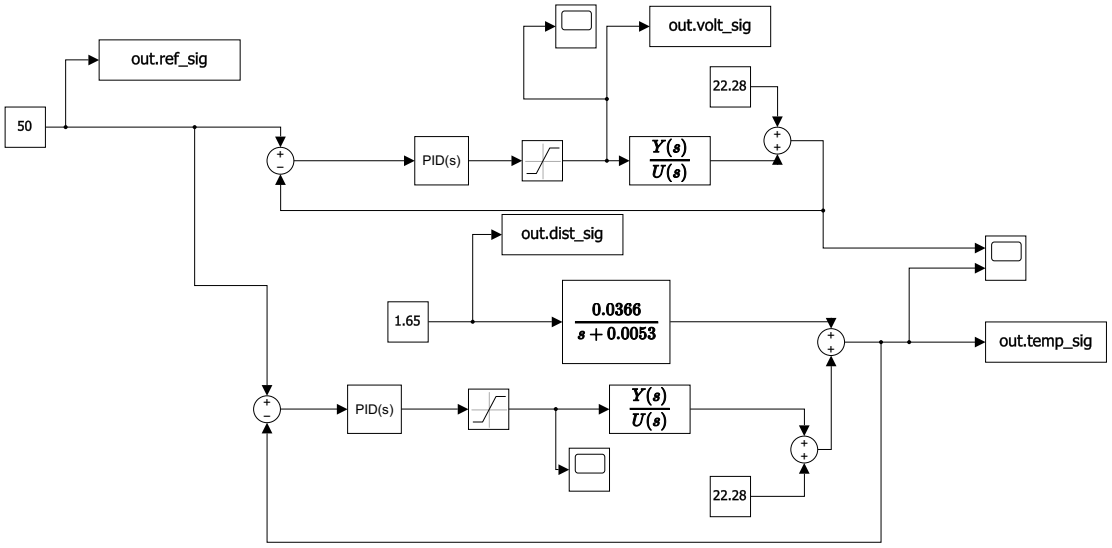


Figure 17: Simulink Model used for PID Disturbance Rejection Controller

Again Matlab's `stepinfo` function was then used to find the following step response parameters.

Step Response Parameter	Closed Loop	Open Loop
Overshoot (%)	9.26	0.0
Rise Time (s)	102.934	291.2
Peak Time (s)	225.625	970.5
Settling Time (s)	366.913	524.4

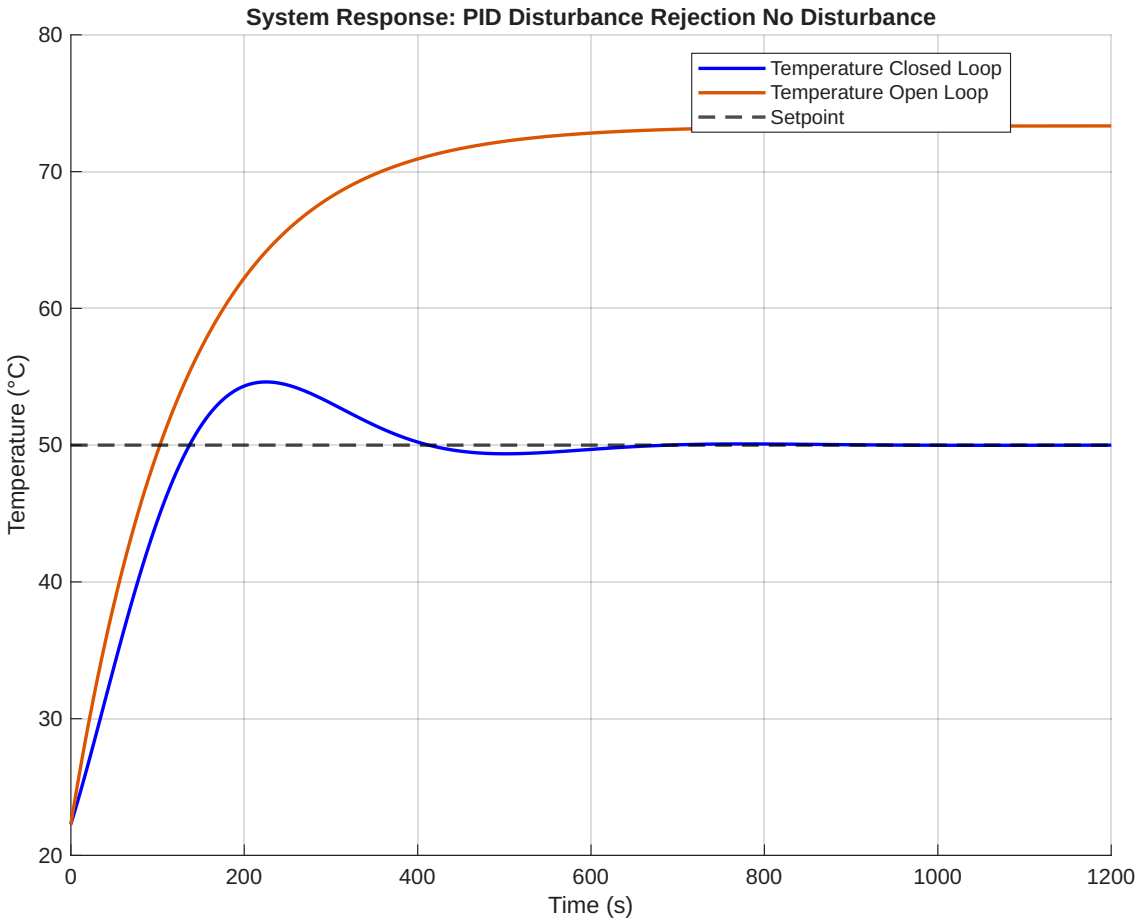
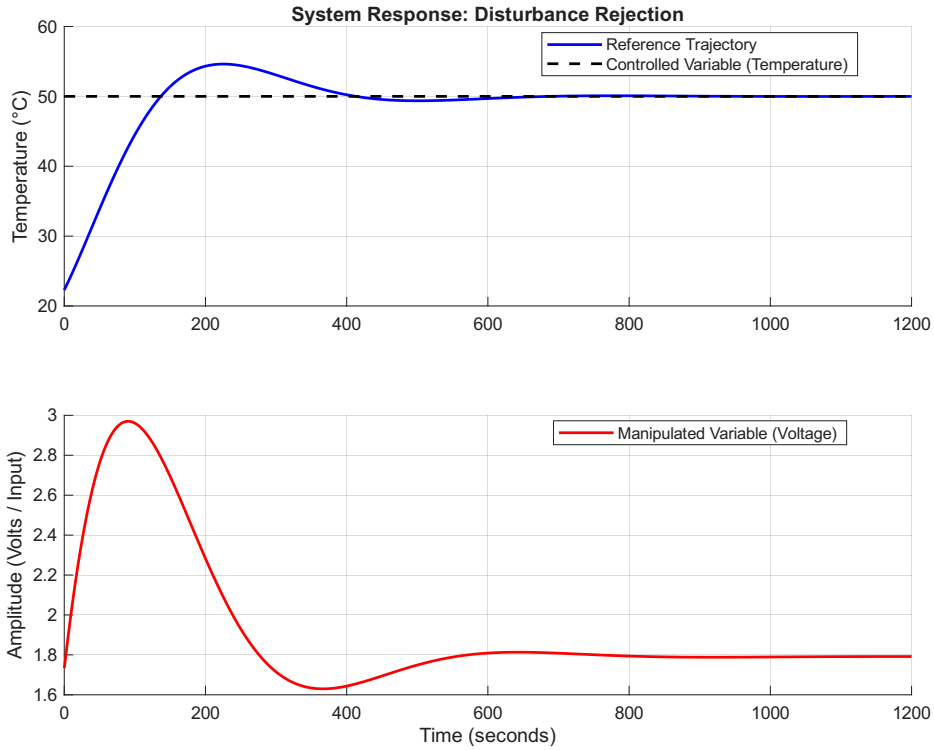
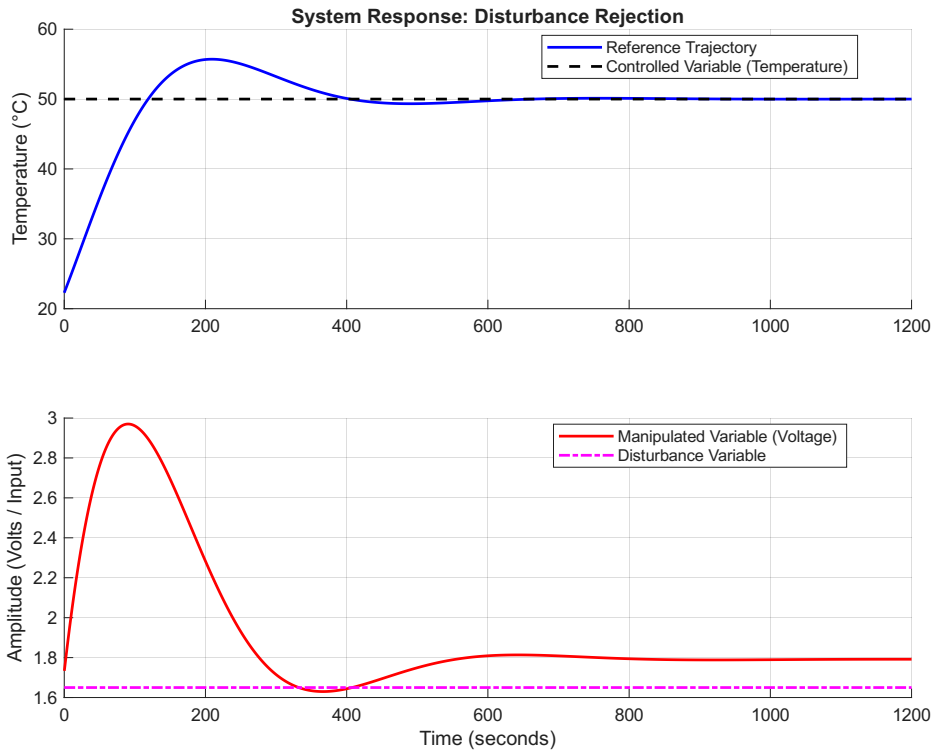


Figure 18: Comparing the closed loop step response of the PID disturbance rejection controller to the open loop step response



(a) Closed-loop response (No Disturbance).



(b) Closed-loop response (50% PWM Disturbance).

Figure 19: Simulink closed-loop step response for the disturbance-rejection PID controller. The bottom figure demonstrates the controller compensating for the external heat disturbance caused by transistor 2.

3 Implementation

3.1 PID Control on MCU

The continuous-time PID controller designed in Simulink was discretized for implementation on the ESP32 microcontroller. The software was developed in C++ and relies on a fixed sample time (Δt) to ensure the integral and derivative calculations remain mathematically accurate.

3.1.1 Discrete PID Algorithm and Hardware Protection

Because the physical plant is constrained by a 0–3.3V operating range mapped to a 0–100% PWM duty cycle, several engineering protections were implemented in the firmware:

- **PID output limiting:** The `pid_out` variable is constrained to the physical limit of 3.3V.
- **Anti-derivative kick:** The first derivative calculation will spike to infinity causing unpredictable behavior, this is handled by implementing a `first_run` flag variable that sets the previous error equal to the current error when the derivative is calculated for the first time.
- **Anti-integral windup:** The integral term does not get updated if the system demands an output higher than the ESP's limit and the error is not negative. The inverse case is handled in the same way.

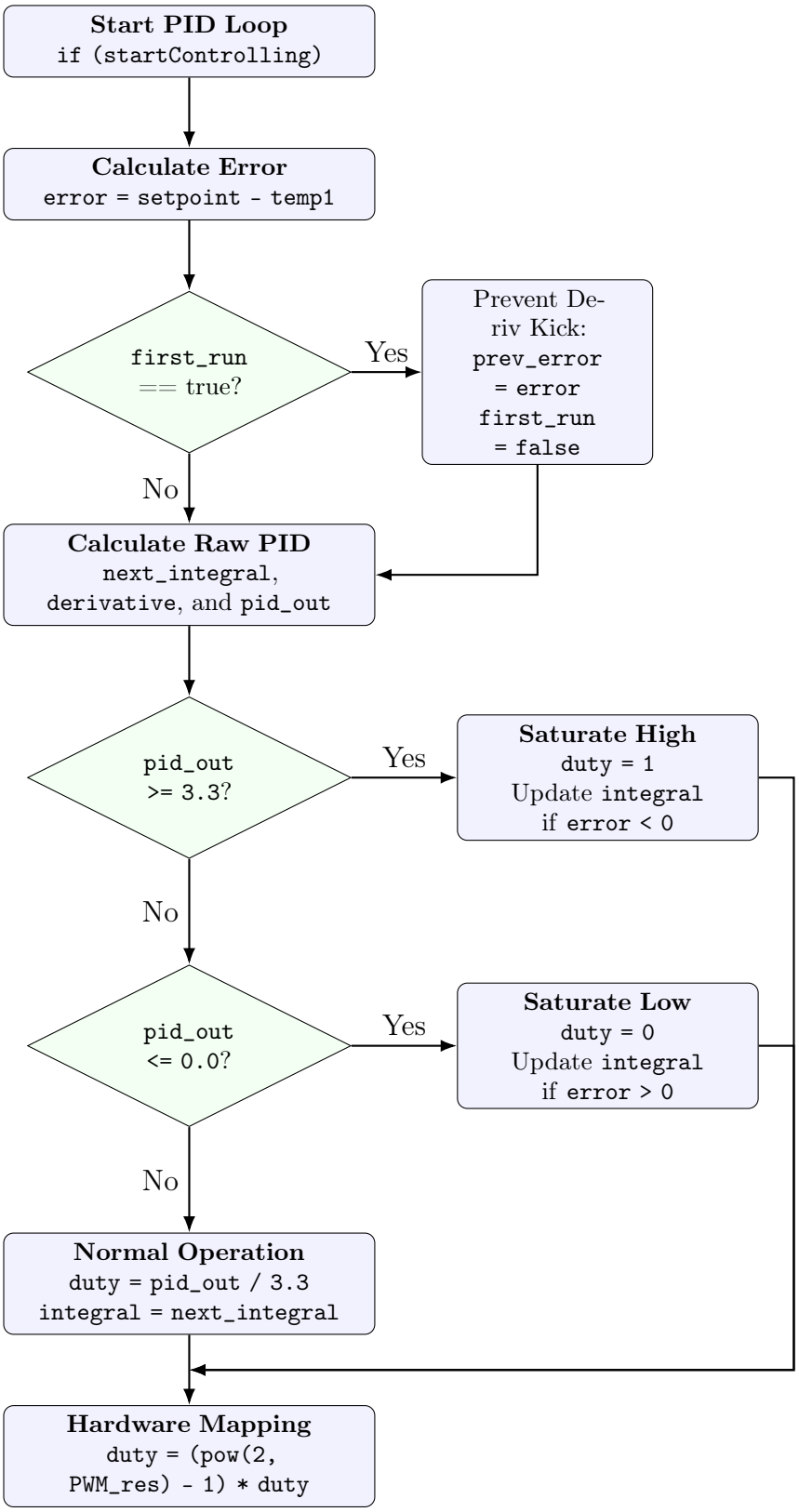


Figure 20: Software execution flow of the discrete PID algorithm, highlighting first-run derivative protection and integral anti-windup logic.

4 Results

4.1 PID Reference Tracking

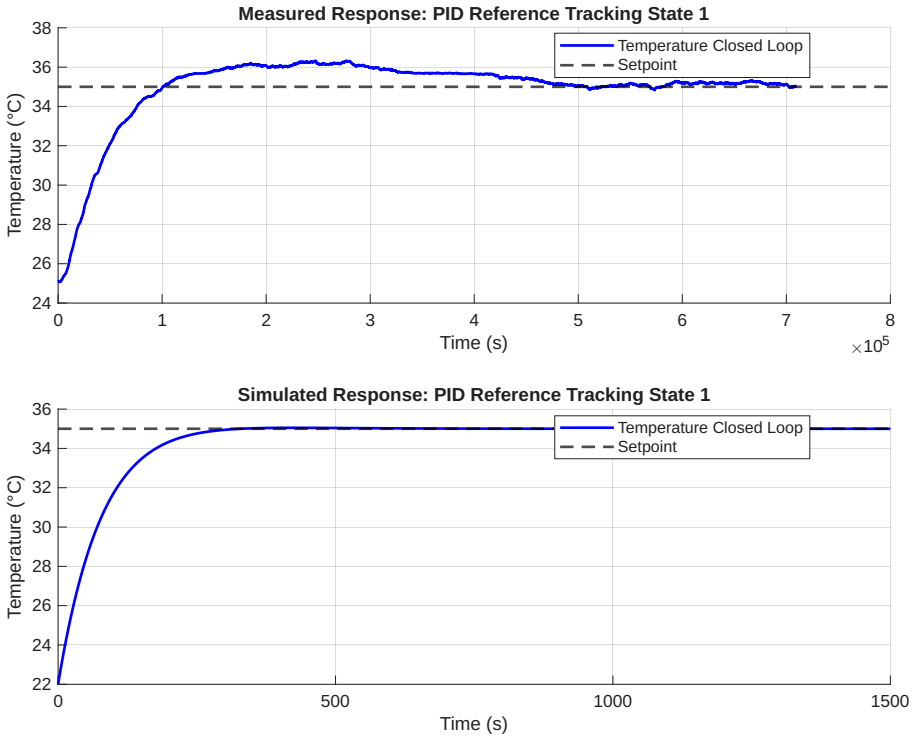


Figure 21: Measured Results for PID reference tracking for state 1

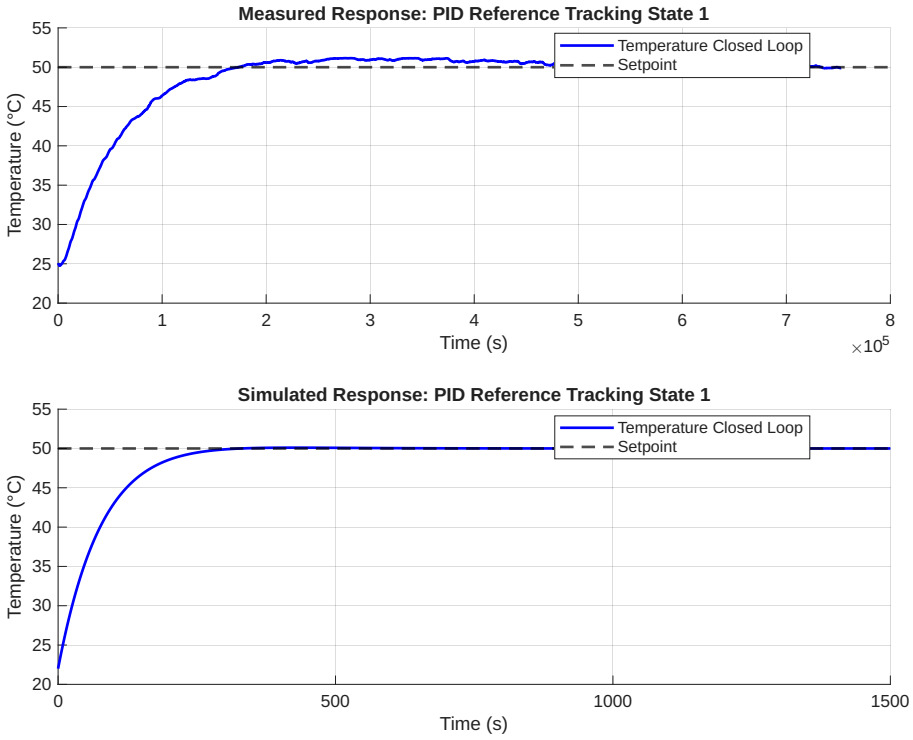


Figure 22: Measured Results for PID reference tracking for state 2

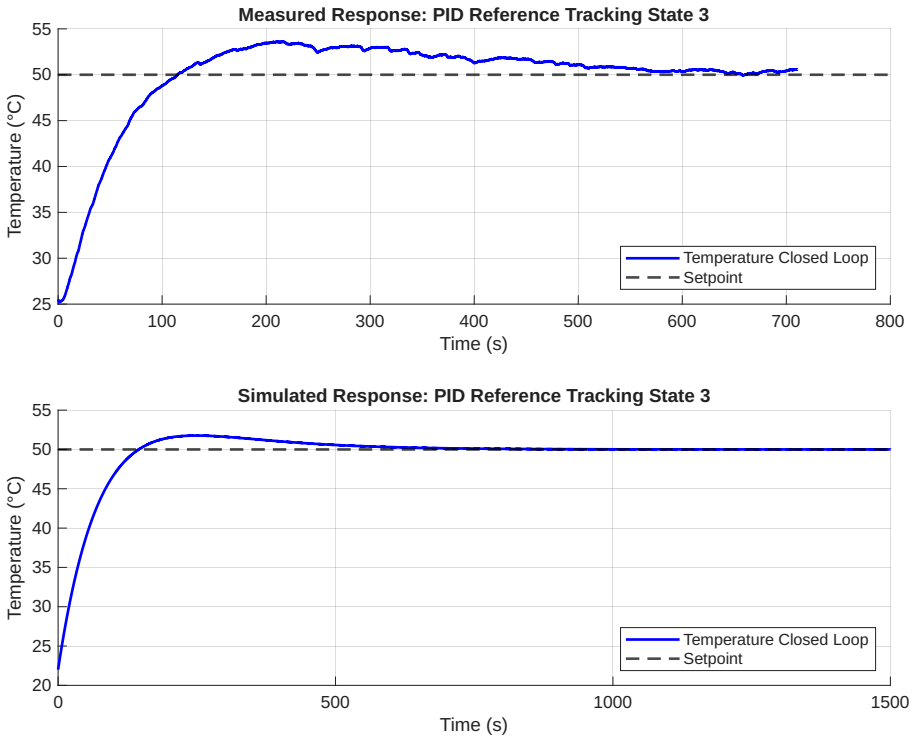


Figure 23: Measured Results for PID reference tracking for state 3

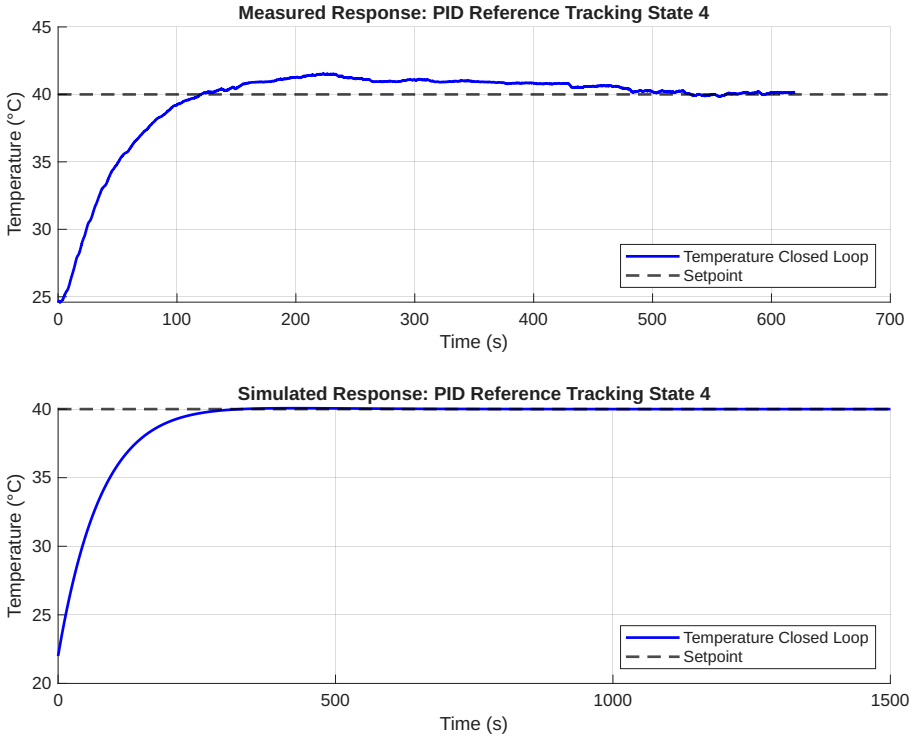


Figure 24: Measured Results for PID reference tracking for state 4

4.2 PID Disturbance Rejection

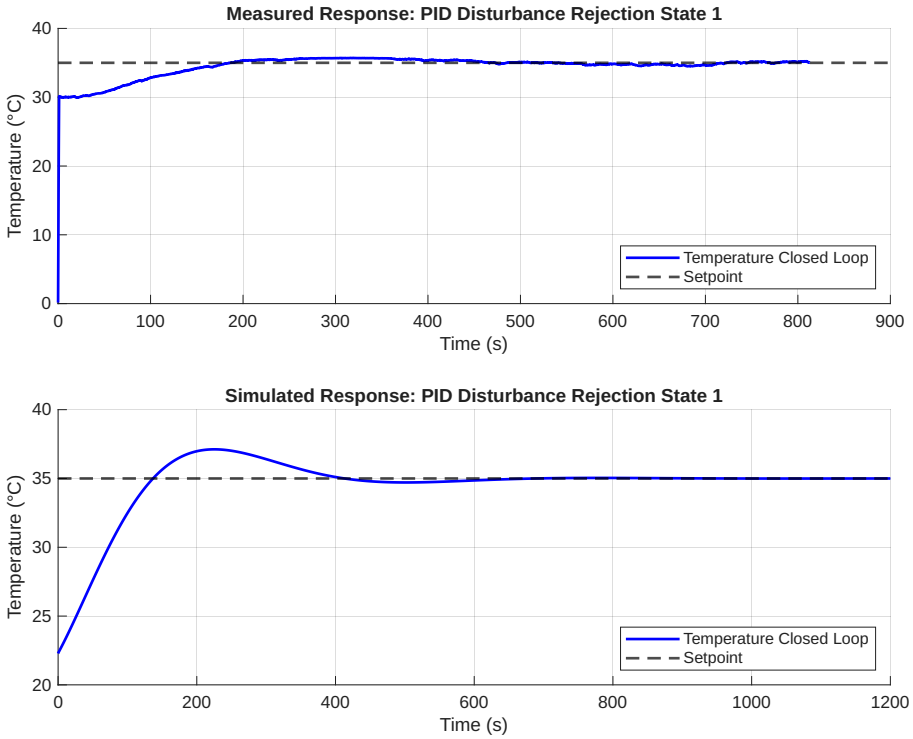


Figure 25: Measured Results for PID disturbance rejection tracking for state 1

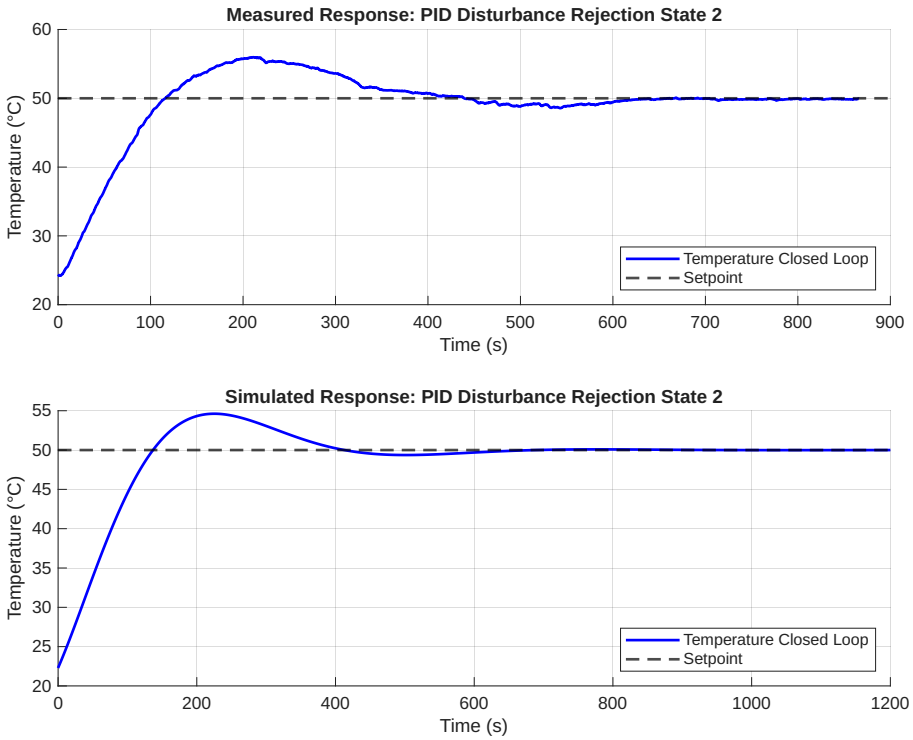


Figure 26: Measured Results for PID disturbance rejection tracking for state 2

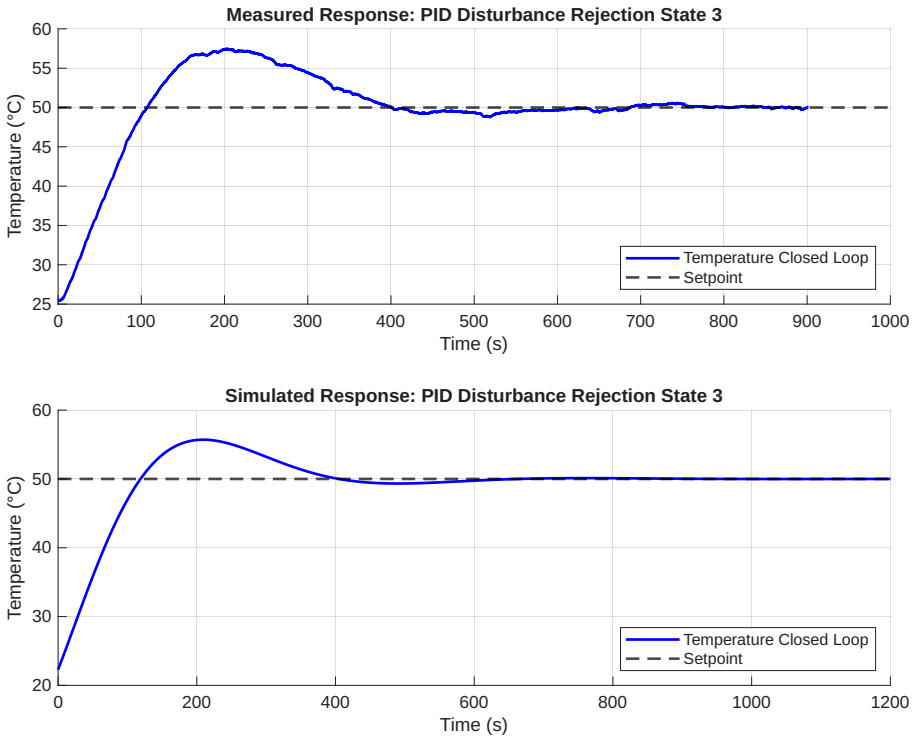


Figure 27: Measured Results for PID disturbance rejection tracking for state 3

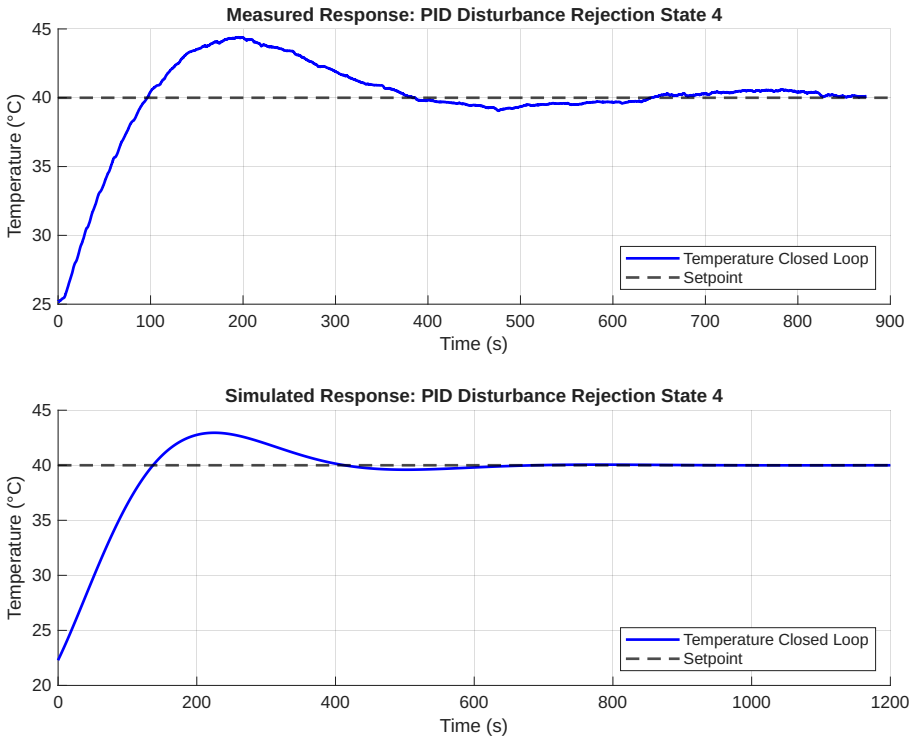


Figure 28: Measured Results for PID disturbance rejection tracking for state 4

5 Discussion

For the most part, the general shape of the plots matched that of the simulations, with the largest exceptions being state one for the disturbance rejection PID and state one for reference tracking PID. It is evident for state one for the disturbance rejection PID that the system was not cooled down to room temperature before the controlling commenced. For state one for the reference tracking PID, various results were found, which are believed to be due to the already weak disturbance rejection causing a significant effect due to external disturbances.

The general PID controller in the Laplace domain is given by:

$$C(s) = K_p + \frac{K_i}{s} + K_d s \tag{4}$$

Each term contributes differently to the closed-loop response, and the simulation results illustrate these effects clearly.

5.1 Proportional Term

The proportional term $u_P(t) = K_p\, e(t)$ produces a control action proportional to the current error. It increases response speed but cannot eliminate steady-state error for a step input. The steady-state output of a P-only closed-loop system is:

$$y_{ss} = \frac{K_p\, G_{11}(0)}{1 + K_p\, G_{11}(0)} = \frac{0.8475 \times 15.59}{1 + 0.8475 \times 15.59} = 0.9296 \tag{5}$$

This matches the simulated result in table 2, where the closed-loop output settles at 92.96% of the 50 °C reference, leaving a permanent error of approximately 3.5 °C as seen in fig. 5. In the disturbance case (fig. 6), the P controller partially compensates by reducing $u(t)$, but a permanent offset remains since there is no mechanism to accumulate and correct the residual error.

5.2 Integral Term

The integral term $u_I(t) = K_i \int_0^t e(\tau)\, d\tau$ accumulates error over time, guaranteeing zero steady-state error for a constant reference by the Internal Model Principle. This is confirmed in table 4, where the PI controller achieves a steady-state gain of exactly 1.0000, and fig. 8 shows $y(t)$ converging to exactly 50 °C.

The cost is a slower, more oscillatory response: rise time increases from 7.9 s (P) to 103.1 s (PI), and overshoot increases from 7.42% to 12.95%. The integral term also fully rejects the constant disturbance, as seen in fig. 9 where $y(t)$ returns to 50 °C after the PWM2 disturbance at $t = 350$ s, unlike the P controller in fig. 6.

5.3 Derivative Term

The derivative term $u_D(t) = K_d \frac{de}{dt}$ acts on the rate of change of error, providing a damping action that reduces overshoot. The PD controller results in table 5 show an overshoot of only 8.8×10^{-5}%, compared to 7.42% for P alone, at the cost of a

slower rise time (77.4 s). Like the P controller, PD retains a steady-state error since it contains no integrator.

5.4 Controller Comparison

Combining all three terms allows the PID controller to benefit from the speed of P, the accuracy of I, and the damping of D simultaneously. The reference-tracking and disturbance-rejection PID controllers (sections 2.4 and 2.5) represent different tuning priorities within this structure: the reference-tracking design favours a well-damped step response, while the disturbance-rejection design increases K_i and K_d to suppress disturbances more aggressively.

6 Code

6.1 Matlab Code

Listing 3: App Code For Sampling

```
%
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%% Auto Generated Code Start
    %%%%%%%%%%%%%%%%%
%
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%


classdef EBBPrac < matlab.apps.AppBase

    % Properties that correspond to app components
    properties (Access = public)
        UIFigure                    matlab.ui.Figure
        GridLayout                  matlab.ui.container.
           GridLayout
        LeftPanel                   matlab.ui.container.
           Panel
        TabGroup                    matlab.ui.container.
           TabGroup
        Tab                         matlab.ui.container.
           Tab
        DataSaveEditField           matlab.ui.control.
           EditField
        DataSaveEditFieldLabel      matlab.ui.control.
           Label
        SamplingtoofastLabel        matlab.ui.control.
           Label
        UpdateChannelsButton        matlab.ui.control.
           Button
        StartButton                 matlab.ui.control.
           StateButton
        StopButton                  matlab.ui.control.
           StateButton
        OutputDropDown              matlab.ui.control.
           DropDown
        OutputDropDownLabel         matlab.ui.control.
           Label
        DutyCycleEditField          matlab.ui.control.
           NumericEditField
        DutyCycleEditFieldLabel     matlab.ui.control.
           Label
```

```
        DisturbanceEditField        matlab.ui.control.
           NumericEditField
        DisturbanceEditFieldLabel   matlab.ui.control.
           Label
        SetpointEditField           matlab.ui.control.
           NumericEditField
        SetpointEditFieldLabel      matlab.ui.control.
           Label
        DEditField                  matlab.ui.control.
           NumericEditField
        DEditFieldLabel             matlab.ui.control.
           Label
        IEditField                  matlab.ui.control.
           NumericEditField
        IEditFieldLabel             matlab.ui.control.
           Label
        PEditField                  matlab.ui.control.
           NumericEditField
        PEditFieldLabel             matlab.ui.control.
           Label
        Tab2                        matlab.ui.container.
           Tab
        BaudRateEditField           matlab.ui.control.
           NumericEditField
        BaudRateEditFieldLabel      matlab.ui.control.
           Label
        COMPortEditField            matlab.ui.control.
           EditField
        COMPortEditFieldLabel       matlab.ui.control.
           Label
        RightPanel                  matlab.ui.container.
           Panel
        UIAxes_2                    matlab.ui.control.
           UIAxes
        UIAxes                      matlab.ui.control.
           UIAxes
    end

    % Properties that correspond to apps with auto-reflow
    properties (Access = private)
        onePanelWidth = 576;
    end
    %
       %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

    %%%%%%%%%%%%%% Auto Generated Code End
       %%%%%%%%%%%%%%%
```

```
%
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

properties (Access = private)
    %Live lines 1 and 2 are for the temperature
       measurements.
    liveLine1
    liveLine2
    %Live lines 3 and 4 are for the PWM signals.
    liveLine3
    liveLine4
    P
    I
    D
    setpoint
    disturbance
    duty_cycle
    output
    capturing
    COM_port
    baud_rate
    window_size
    esp32
    data_save_name % Description
end


% Callbacks that handle component events
methods (Access = private)

    % Value changed function: PEditField
    function PEditFieldValueChanged(app, event)
        app.P = app.PEditField.Value;
    end

    % Value changed function: IEditField
    function IEditFieldValueChanged(app, event)
        app.I = app.IEditField.Value;
    end

    % Value changed function: DEditField
    function DEditFieldValueChanged(app, event)
        app.D = app.DEditField.Value;
    end

    % Value changed function: SetpointEditField
```

```
function SetpointEditFieldValueChanged(app, event
    )
    app.setpoint = app.SetpointEditField.Value;
end

% Value changed function: DisturbanceEditField
function DisturbanceEditFieldValueChanged(app,
    event)
    app.disturbance = app.DisturbanceEditField.
        Value;
end

% Value changed function: DutyCycleEditField
function DutyCycleEditFieldValueChanged(app,
    event)
    app.duty_cycle = app.DutyCycleEditField.Value
        ;
end

% Value changed function: OutputDropDown
function OutputDropDownValueChanged(app, event)
    app.output = str2double(app.OutputDropDown.
        Value);
end

% Value changed function: StartButton
function StartButtonValueChanged(app, event)
    app.StartButton.Visible = "off";
    app.StopButton.Visible = "on";
    app.UpdateChannelsButton.Visible = "on";
    app.capturing = true;

    %Call all of the other functions to make sure
        their values are
    %captured.
    app.PEditFieldValueChanged();
    app.IEditFieldValueChanged();
    app.DEditFieldValueChanged();
    app.SetpointEditFieldValueChanged();
    app.DisturbanceEditFieldValueChanged();
    app.DutyCycleEditFieldValueChanged();
    app.OutputDropDownValueChanged();
    app.BaudRateEditFieldValueChanged();
    app.COMPortEditFieldValueChanged();
    app.DataSaveEditFieldValueChanged();

    if isempty(app.esp32)
```

```
        app.esp32 = serialport(app.COM_port, app.
            baud_rate);
        configureTerminator(app.esp32, "LF");
    end
    flush(app.esp32);

    pause(1);

    %Make sure all info is read into esp.
    writeline(app.esp32, sprintf("%d, %.2f", 0,
        app.P)); %
    writeline(app.esp32, sprintf("%d, %.2f", 1,
        app.I)); %
    writeline(app.esp32, sprintf("%d, %.2f", 2,
        app.D)); %
    writeline(app.esp32, sprintf("%d, %.2f", 3,
        app.setpoint)); %
    writeline(app.esp32, sprintf("%d, %.2f", 4,
        app.disturbance)); %
    writeline(app.esp32, sprintf("%d, %d", 5, app
        .duty_cycle)); %
    writeline(app.esp32, sprintf("%d, %.2f", 6,
        app.output)); %
    writeline(app.esp32, sprintf("%d, %.2f", 7,
        app.capturing)); %

    %Clear axes.
    cla(app.UIAxes);
    cla(app.UIAxes_2);

    %Plotting axes for the temperature
        measurements.
    app.liveLine1 = animatedline(app.UIAxes, '
        Color', 'r');
    app.liveLine2 = animatedline(app.UIAxes, '
        Color', 'b');

    %Plotting axes for the PWM signals.
    app.liveLine3 = animatedline(app.UIAxes_2, '
        Color', 'r');
    app.liveLine4 = animatedline(app.UIAxes_2, '
        Color', 'b');

    app.window_size = 30; %seconds

    %Define data csv that we will safe to.
    fid = fopen(app.data_save_name + ".csv", 'wt'
        );
```

```
while(app.capturing)
    if app.esp32.NumBytesAvailable > 0
        rawData = readline(app.esp32);
        data = str2double(split(rawData, ',')
           );
        data(5) = data(5)/1000;

        if ~isnan(data(1)) && (length(data)
           == 6)
            %Plot temperature data.
            addpoints(app.liveLine1, data(5),
                data(1));
            addpoints(app.liveLine2, data(5),
                data(2));
            xlim(app.UIAxes, [data(5) - app.
               window_size, data(5)]);

            %Plot PWM signals.
            addpoints(app.liveLine3, data(5),
                data(3));
            addpoints(app.liveLine4, data(5),
                data(4));
            xlim(app.UIAxes_2, [data(5) - app
               .window_size, data(5)]);

            %Write data to file.
            fprintf(fid, rawData);

            if data(end) == 0
                app.SamplingtoofastLabel.
                   Visible = "on";
            else
                app.SamplingtoofastLabel.
                   Visible = "off";
            end
        end

        drawnow limitrate
    end
end

%Close file we are saving to.
fclose(fid);

%Send stop signal.
writeline(app.esp32, sprintf("%d, %.2f", 7,
   app.capturing));
```

```
    clear app.esp32;
end

% Value changed function: COMPortEditField
function COMPortEditFieldValueChanged(app, event)
    app.COM_port = app.COMPortEditField.Value;
end

% Value changed function: BaudRateEditField
function BaudRateEditFieldValueChanged(app, event
    )
    app.baud_rate = app.BaudRateEditField.Value;
end

% Value changed function: StopButton
function StopButtonValueChanged(app, event)
    app.capturing = false;
    app.StopButton.Visible = "off";
    app.UpdateChannelsButton.Visible = "off";
    app.StartButton.Visible = "on";

end

% Button pushed function: UpdateChannelsButton
function UpdateChannelsButtonPushed(app, event)
    app.DutyCycleEditFieldValueChanged();
    app.OutputDropDownValueChanged();
    writeline(app.esp32, sprintf("%d, %.2f", 5,
        app.duty_cycle));
    writeline(app.esp32, sprintf("%d, %.2f", 6,
        app.output));
end

% Value changed function: DataSaveEditField
function DataSaveEditFieldValueChanged(app, event
    )
    app.data_save_name = app.DataSaveEditField.
        Value;

end

% Changes arrangement of the app based on
    UIFigure width
function updateAppLayout(app, event)
    currentFigureWidth = app.UIFigure.Position(3)
        ;
    if(currentFigureWidth <= app.onePanelWidth)
```

```
            % Change to a 2x1 grid
            app.GridLayout.RowHeight = {449, 449};
            app.GridLayout.ColumnWidth = {'1x'};
            app.RightPanel.Layout.Row = 2;
            app.RightPanel.Layout.Column = 1;
        else
            % Change to a 1x2 grid
            app.GridLayout.RowHeight = {'1x'};
            app.GridLayout.ColumnWidth = {204, '1x'};
            app.RightPanel.Layout.Row = 1;
            app.RightPanel.Layout.Column = 2;
        end
    end
end

% %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%% Auto Generated Code Start %%%%%%%%%%%%
% %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%


% Component initialization
methods (Access = private)

    % Create UIFigure and components
    function createComponents(app)

        % Create UIFigure and hide until all
           components are created
        app.UIFigure = uifigure('Visible', 'off');
        app.UIFigure.AutoResizeChildren = 'off';
        app.UIFigure.Position = [100 100 702 449];
        app.UIFigure.Name = 'MATLAB App';
        app.UIFigure.SizeChangedFcn =
           createCallbackFcn(app, @updateAppLayout,
           true);

        % Create GridLayout
        app.GridLayout = uigridlayout(app.UIFigure);
        app.GridLayout.ColumnWidth = {204, '1x'};
        app.GridLayout.RowHeight = {'1x'};
        app.GridLayout.ColumnSpacing = 0;
        app.GridLayout.RowSpacing = 0;
        app.GridLayout.Padding = [0 0 0 0];
        app.GridLayout.Scrollable = 'on';
```

```
% Create LeftPanel
app.LeftPanel = uipanel(app.GridLayout);
app.LeftPanel.Layout.Row = 1;
app.LeftPanel.Layout.Column = 1;

% Create TabGroup
app.TabGroup = uitabgroup(app.LeftPanel);
app.TabGroup.Position = [3 3 197 442];

% Create Tab
app.Tab = uitab(app.TabGroup);
app.Tab.Title = 'Tab';

% Create PEditFieldLabel
app.PEditFieldLabel = uilabel(app.Tab);
app.PEditFieldLabel.HorizontalAlignment = '
   right';
app.PEditFieldLabel.Position = [46 385 25
   22];
app.PEditFieldLabel.Text = 'P';

% Create PEditField
app.PEditField = uieditfield(app.Tab, '
   numeric');
app.PEditField.ValueChangedFcn =
   createCallbackFcn(app, @
   PEditFieldValueChanged, true);
app.PEditField.Position = [85 385 100 22];

% Create IEditFieldLabel
app.IEditFieldLabel = uilabel(app.Tab);
app.IEditFieldLabel.HorizontalAlignment = '
   right';
app.IEditFieldLabel.Position = [46 355 25
   22];
app.IEditFieldLabel.Text = 'I';

% Create IEditField
app.IEditField = uieditfield(app.Tab, '
   numeric');
app.IEditField.ValueChangedFcn =
   createCallbackFcn(app, @
   IEditFieldValueChanged, true);
app.IEditField.Position = [85 355 100 22];

% Create DEditFieldLabel
app.DEditFieldLabel = uilabel(app.Tab);
```

```
app.DEditFieldLabel.HorizontalAlignment = '
    right';
app.DEditFieldLabel.Position = [46 326 25
    22];
app.DEditFieldLabel.Text = 'D';

% Create DEditField
app.DEditField = uieditfield(app.Tab, '
    numeric');
app.DEditField.ValueChangedFcn =
    createCallbackFcn(app, @
    DEditFieldValueChanged, true);
app.DEditField.Position = [85 326 100 22];

% Create SetpointEditFieldLabel
app.SetpointEditFieldLabel = uilabel(app.Tab)
    ;
app.SetpointEditFieldLabel.
    HorizontalAlignment = 'right';
app.SetpointEditFieldLabel.Position = [21 296
     49 22];
app.SetpointEditFieldLabel.Text = 'Setpoint';

% Create SetpointEditField
app.SetpointEditField = uieditfield(app.Tab,
    'numeric');
app.SetpointEditField.ValueChangedFcn =
    createCallbackFcn(app, @
    SetpointEditFieldValueChanged, true);
app.SetpointEditField.Position = [85 296 100
    22];

% Create DisturbanceEditFieldLabel
app.DisturbanceEditFieldLabel = uilabel(app.
    Tab);
app.DisturbanceEditFieldLabel.
    HorizontalAlignment = 'right';
app.DisturbanceEditFieldLabel.Position = [1
    265 69 22];
app.DisturbanceEditFieldLabel.Text = '
    Disturbance';

% Create DisturbanceEditField
app.DisturbanceEditField = uieditfield(app.
    Tab, 'numeric');
app.DisturbanceEditField.ValueChangedFcn =
    createCallbackFcn(app, @
    DisturbanceEditFieldValueChanged, true);
```

```
app.DisturbanceEditField.Position = [85 265
    100 22];

% Create DutyCycleEditFieldLabel
app.DutyCycleEditFieldLabel = uilabel(app.Tab
    );
app.DutyCycleEditFieldLabel.
    HorizontalAlignment = 'right';
app.DutyCycleEditFieldLabel.Position = [9 235
     63 22];
app.DutyCycleEditFieldLabel.Text = 'Duty
    Cycle';

% Create DutyCycleEditField
app.DutyCycleEditField = uieditfield(app.Tab,
     'numeric');
app.DutyCycleEditField.ValueChangedFcn =
    createCallbackFcn(app, @
    DutyCycleEditFieldValueChanged, true);
app.DutyCycleEditField.Position = [86 235 100
     22];
app.DutyCycleEditField.Value = 100;

% Create OutputDropDownLabel
app.OutputDropDownLabel = uilabel(app.Tab);
app.OutputDropDownLabel.HorizontalAlignment =
     'right';
app.OutputDropDownLabel.Position = [30 199 41
     22];
app.OutputDropDownLabel.Text = 'Output';

% Create OutputDropDown
app.OutputDropDown = uidropdown(app.Tab);
app.OutputDropDown.Items = {'None', 'Channel1
    ', 'Channel2', 'Both'};
app.OutputDropDown.ItemsData = {'3', '0', '1'
    , '2'};
app.OutputDropDown.ValueChangedFcn =
    createCallbackFcn(app, @
    OutputDropDownValueChanged, true);
app.OutputDropDown.Position = [85 199 100
    22];
app.OutputDropDown.Value = '3';

% Create StopButton
app.StopButton = uibutton(app.Tab, 'state');
app.StopButton.ValueChangedFcn =
    createCallbackFcn(app, @
```

```
    StopButtonValueChanged, true);
app.StopButton.Visible = 'off';
app.StopButton.Text = 'Stop';
app.StopButton.Position = [85 168 100 23];

% Create StartButton
app.StartButton = uibutton(app.Tab, 'state');
app.StartButton.ValueChangedFcn =
    createCallbackFcn(app, @
    StartButtonValueChanged, true);
app.StartButton.Text = 'Start';
app.StartButton.Position = [86 168 100 23];

% Create UpdateChannelsButton
app.UpdateChannelsButton = uibutton(app.Tab,
    'push');
app.UpdateChannelsButton.ButtonPushedFcn =
    createCallbackFcn(app, @
    UpdateChannelsButtonPushed, true);
app.UpdateChannelsButton.Visible = 'off';
app.UpdateChannelsButton.Position = [81 136
    108 23];
app.UpdateChannelsButton.Text = 'Update
    Channels';

% Create SamplingtoofastLabel
app.SamplingtoofastLabel = uilabel(app.Tab);
app.SamplingtoofastLabel.Visible = 'off';
app.SamplingtoofastLabel.Position = [90 12
    101 22];
app.SamplingtoofastLabel.Text = 'Sampling too
     fast!';

% Create DataSaveEditFieldLabel
app.DataSaveEditFieldLabel = uilabel(app.Tab)
    ;
app.DataSaveEditFieldLabel.
    HorizontalAlignment = 'right';
app.DataSaveEditFieldLabel.Position = [10 96
    61 22];
app.DataSaveEditFieldLabel.Text = 'Data Save'
    ;

% Create DataSaveEditField
app.DataSaveEditField = uieditfield(app.Tab,
    'text');
app.DataSaveEditField.ValueChangedFcn =
    createCallbackFcn(app, @
```

```
    DataSaveEditFieldValueChanged, true);
app.DataSaveEditField.Position = [86 96 100
    22];
app.DataSaveEditField.Value = 'PWM1StepV0';

% Create Tab2
app.Tab2 = uitab(app.TabGroup);
app.Tab2.Title = 'Tab2';

% Create COMPortEditFieldLabel
app.COMPortEditFieldLabel = uilabel(app.Tab2)
    ;
app.COMPortEditFieldLabel.HorizontalAlignment
     = 'right';
app.COMPortEditFieldLabel.Position = [13 385
    58 22];
app.COMPortEditFieldLabel.Text = 'COM Port';

% Create COMPortEditField
app.COMPortEditField = uieditfield(app.Tab2,
    'text');
app.COMPortEditField.ValueChangedFcn =
    createCallbackFcn(app, @
    COMPortEditFieldValueChanged, true);
app.COMPortEditField.Position = [85 385 100
    22];
app.COMPortEditField.Value = '/dev/ttyUSB0';

% Create BaudRateEditFieldLabel
app.BaudRateEditFieldLabel = uilabel(app.Tab2
    );
app.BaudRateEditFieldLabel.
    HorizontalAlignment = 'right';
app.BaudRateEditFieldLabel.Position = [9 355
    62 22];
app.BaudRateEditFieldLabel.Text = 'Baud Rate'
    ;

% Create BaudRateEditField
app.BaudRateEditField = uieditfield(app.Tab2,
     'numeric');
app.BaudRateEditField.ValueChangedFcn =
    createCallbackFcn(app, @
    BaudRateEditFieldValueChanged, true);
app.BaudRateEditField.Position = [85 355 100
    22];
app.BaudRateEditField.Value = 115200;
```

```
            % Create RightPanel
            app.RightPanel = uipanel(app.GridLayout);
            app.RightPanel.Layout.Row = 1;
            app.RightPanel.Layout.Column = 2;

            % Create UIAxes
            app.UIAxes = uiaxes(app.RightPanel);
            title(app.UIAxes, 'Title')
            xlabel(app.UIAxes, 'X')
            ylabel(app.UIAxes, 'Y')
            zlabel(app.UIAxes, 'Z')
            app.UIAxes.Position = [7 213 489 235];

            % Create UIAxes_2
            app.UIAxes_2 = uiaxes(app.RightPanel);
            title(app.UIAxes_2, 'Title')
            xlabel(app.UIAxes_2, 'X')
            ylabel(app.UIAxes_2, 'Y')
            zlabel(app.UIAxes_2, 'Z')
            app.UIAxes_2.Position = [8 7 488 207];

            % Show the figure after all components are
               created
            app.UIFigure.Visible = 'on';
        end
    end

    % App creation and deletion
    methods (Access = public)

        % Construct app
        function app = EBBPrac

            % Create UIFigure and components
            createComponents(app)

            % Register the app with App Designer
            registerApp(app, app.UIFigure)

            if nargout == 0
                clear app
            end
        end

        % Code that executes before app deletion
        function delete(app)

            % Delete UIFigure when app is deleted
```

```
            delete(app.UIFigure)
        end
    end
end
%
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%%%%%%%% Auto Generated Code end
    %%%%%%%%%%%%%
%
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

6.2 Model Fit Code

Listing 4: Model fitting and plotting code

```
% Obtaining the transfer function
% Package the input arrays of both transistors into one
   matrix (Needs to be columns)
PWM1StepV1Data = readmatrix('..\Data1\Step_PWM1_V0.csv');
PWM1StepV1 = PWM1StepV1Data(7920:end,1:5);
PWM2StepV0Data = readmatrix('..\Data1\Step_PWM2_V0.csv');
PWM2StepV0 = PWM2StepV0Data(6400:end,1:5);

PWM1TestDataFile = readmatrix('..\Data1\
   Step_PWM1_VtestData50.csv');
PWM1TestData = PWM1TestDataFile(6230:end,1:5);
PWM2TestDataFile = readmatrix('..\Data1\
   Step_PWM2_VtestData50.csv');
PWM2TestData = PWM2TestDataFile(6160:end,1:5);

% pwm1 data prep
V_PWM1 = [PWM1StepV1(:,3)];
Temp_PWM1 = [PWM1StepV1(:,1)-22.28, PWM1StepV1(:,2)
   -21.24];
time_step = 0.005;

% pwm2 data prep
V_PWM2 = [PWM2StepV0(:,4)];
Temp_PWM2 = [PWM2StepV0(:,1)-20.26, PWM2StepV0(:,2)
   -19.05];

% pwm1 test data prep
V_PWM1_Test = [PWM1TestData(:,3)];
Temp_PWM1_test = [PWM1TestData(:,1)-21.2, PWM1TestData
   (:,2)-20.2];

% pwm2 test data prep
```

```
V_PWM2_Test = [PWM2TestData(:,4)];
Temp_PWM2_test = [PWM2TestData(:,1)-21.1, PWM2TestData
   (:,2)-20.2] ;

% use iddata to set up the data for tfest
tfest_data1 = iddata(Temp_PWM1, V_PWM1, time_step);
tfest_data2 = iddata(Temp_PWM2, V_PWM2, time_step);
testData1 = iddata(Temp_PWM1_test, V_PWM1_Test, time_step
   );
testData2 = iddata(Temp_PWM2_test, V_PWM2_Test, time_step
   );

% estimate nr of poles and zeros
n_poles = 1;
n_zeros = 0;

% Estimate the transfer functions independently (SISO) to
    avoid idss error
% tfest_data(samples, output, input)
sys11 = tfest(tfest_data1(:, 1, 1), n_poles, n_zeros, NaN
   ); % PWM1 to T1
sys21 = tfest(tfest_data1(:, 2, 1), n_poles, n_zeros, NaN
   ); % PWM1 to T2

sys12 = tfest(tfest_data2(:, 1, 1), n_poles, n_zeros, NaN
   ); % PWM2 to T1
sys22 = tfest(tfest_data2(:, 2, 1), n_poles, n_zeros, NaN
   ); % PWM2 to T2

% Combine the individual SISO models back into SIMO
   systems
simo_transfer1 = [sys11; sys21];
simo_transfer2 = [sys12; sys22];

% set up graph details
tfest_data1.Name = 'Measured Data';
tfest_data2.Name = 'Measured Data';
testData1.Name = 'Measured Data';
testData2.Name = 'Measured Data';
simo_transfer2.Name = 'Simulated Model';
simo_transfer1.Name = 'Simulated Model';

figure(1)
compare(tfest_data1, simo_transfer1);
legend('Location', 'best');
ylabel('Change in Temperature (C)')

figure(2)
```

```
compare(tfest_data2, simo_transfer2);
legend('Location', 'best');
ylabel('Change in Temperature (C)')

figure(3)
compare(testData1, simo_transfer1);
legend('Location', 'best');
ylabel('Change in Temperature (C)')

figure(4)
compare(testData2, simo_transfer2);
legend('Location', 'best');
ylabel('Change in Temperature (C)')

lines = findobj(gcf, 'Type', 'Line');
set(lines, 'LineWidth', 1.5);

% display the transfer functions of PWM1
[num1, den1] = tfdata(simo_transfer1(1, 1), 'v');
delay1 = simo_transfer1(1, 1).IODelay; % Extract dead
   time
[num2, den2] = tfdata(simo_transfer1(2, 1), 'v');
delay2 = simo_transfer1(2, 1).IODelay;

disp('--- Transfer Function 1 (PWM1 to T1) ---');
disp('Numerator:'); disp(num1);
disp('Denominator:'); disp(den1);
disp(['Estimated Dead Time (s): ', num2str(delay1)]);

disp('--- Transfer Function 2 (PWM1 to T2) ---');
disp('Numerator:'); disp(num2);
disp('Denominator:'); disp(den2);
disp(['Estimated Dead Time (s): ', num2str(delay2)]);

% display transfer functions of PWM2
[num12, den12] = tfdata(simo_transfer2(1, 1), 'v');
delay12 = simo_transfer2(1, 1).IODelay;
[num22, den22] = tfdata(simo_transfer2(2, 1), 'v');
delay22 = simo_transfer2(2, 1).IODelay;

disp('--- Transfer Function 1 (PWM2 to T1) ---');
disp('Numerator:'); disp(num12);
disp('Denominator:'); disp(den12);
disp(['Estimated Dead Time (s): ', num2str(delay12)]);

disp('--- Transfer Function 2 (PWM2 to T2) ---');
disp('Numerator:'); disp(num22);
disp('Denominator:'); disp(den22);
```

```
disp(['Estimated Dead Time (s): ', num2str(delay22)]);
```

6.3 Prac 2 — Controller Design (P and PI)

6.4 Prac 2 — Report Figure Generation (P and PI)

6.5 MCU Code

```
#include <fmt/core.h>
#include <Arduino.h>
#include <iostream>
#include <sstream>
#include <vector>
#include <string>
#include <queue>

double P = 0;
double I = 0;
double D = 0;
double setpoint = 0;
double disturbance = 0;
double duty = 128; //50% duty cycle by default.
int control_outputs = 0; //By default only pwm1 is on.
bool start = 0; //Start off.
int message = 100; //Flags for certain messages.
int start_time = 0;
bool startControlling = 0; //See if we should start
    controlling, or continue just measuring.

//PID variables
float error = 0;
float integral = 0;
float previous_error = 0;
float derivative = 0;
bool do_integration = true;



double temp1 = 0;
double temp2 = 0;
int N_samples = 200;
std::queue<double> samples1;
std::queue<double> samples2;

hw_timer_t *timer = NULL;
const int PWM_Out_1 = GPIO_NUM_32;
const int PWM_Out_2 = GPIO_NUM_25;
const int PWM_Meas_1 = GPIO_NUM_33;
const int PWM_Meas_2 = GPIO_NUM_26;
```

```
const int Temp_Meas1 = GPIO_NUM_27;
const int Temp_Meas2 = GPIO_NUM_35;

const double A = (3.3/4095.0);
const int PWM_freq = 10;
const int PWM_res = 8;

const int BaudRate = 1152000;
const int sampling_period = 5000; //In microseconds.
const float dt = sampling_period/1000000.0;
bool first_run = true;

void setup() {
  //Start the USB-C serial/UART port.
  Serial.begin(BaudRate);

  //Setup I/O pins.
  pinMode(Temp_Meas1, INPUT);
  pinMode(Temp_Meas2, INPUT);
  pinMode(PWM_Meas_1, INPUT);
  pinMode(PWM_Meas_2, INPUT);

  ledcChangeFrequency(0, PWM_freq, PWM_res);
  ledcChangeFrequency(1, PWM_freq, PWM_res);
  ledcAttachPin (PWM_Out_1, 0);
  ledcAttachPin(PWM_Out_2, 1);

  timer = timerBegin(0, 80, true);

  samples1.push(0);
  samples2.push(0);
}

void loop() {

  //If receiving communication from computer, update values.
  if(Serial.available() > 0)
  {
    String received = Serial.readStringUntil('\n');
    received.trim();

    std::istringstream stream(received.c_str());
    std::string command;
    std::string value_str;
    double value;

    std::getline(stream, command, ',');
```

```
    std::getline(stream, value_str, ',');
    value = std::stod(value_str);

    if(command == "0")
    {
      P = value;
    }
    else if(command == "1")
    {
      I = value;
    }
    else if(command == "2")
    {
      D = value;
    }
    else if(command == "3")
    {
      setpoint = value;
    }
    else if(command == "4")
    {
      disturbance = value;
      disturbance = (pow(2, PWM_res) - 1)*(disturbance
         /100.0);
    }
    else if(command == "5") //Change from duty cycle set to
       the startControlling flag.
    {
      startControlling = value;
    }
    else if(command == "6")
    {
      control_outputs = value;
      //Serial.print("Igot");
      //Serial.println(control_outputs);
    }
    else if(command == "7")
    {
      start = value;

      //Check if start is true, then start timer.
      if(start)
      {
        start_time = millis();
      }
    }
}
```

```
//If start flag is true, start measuring inputs/start
    controlling aswell.
if(start)
{
  //Check when we started in order to see how long we were
      busy.
  timerRestart(timer);

  if(startControlling)
  {
    //Turn on the appropriate PWM signals.
      ledcWrite(0, duty);
      ledcWrite(1, disturbance);
  }

  //Interpret sensor data.
  double temp1_raw = (analogReadMilliVolts(Temp_Meas1)
     /1000.0 - 0.5)/0.01;
  double temp2_raw = (analogReadMilliVolts(Temp_Meas2)
     /1000.0 - 0.5)/0.01;

  if(samples1.size() > N_samples)
  {
    temp1 -= samples1.front();
    temp2 -= samples2.front();

    samples1.pop();
    samples2.pop();
  }

  temp1 += temp1_raw/N_samples;
  temp2 += temp2_raw/N_samples;

  samples1.push(temp1_raw/N_samples);
  samples2.push(temp2_raw/N_samples);

  //Send sensor input data as a comma seperated list.
  std::string sending = fmt::format("{0}, {1}, {2}, {3},
     {4}, {5}",
  temp1, temp2, analogRead(PWM_Meas_1)*A, analogRead(
     PWM_Meas_2)*A, millis() - start_time, message);

  Serial.println(String(sending.c_str()));
  Serial.flush();


  if (startControlling)
  {
```

```
  //PID controller start
  previous_error = error;
  error = setpoint - temp1;

  // Prevent derivative kick on first run

  if (first_run) {
    previous_error = error;
    first_run = false;
  }

  float next_integral = integral + error * dt;

  derivative = (error - previous_error)/dt;

  float pid_out = P * error + I*next_integral + D*
     derivative;
  //anti-windup logic
  if  (pid_out >= 3.3 )
  {
    duty = 1;
    if ( error < 0){
    integral = next_integral;
    }

  }

  else if (pid_out <= 0.0 ) {
  duty = 0.0; // You cannot have a negative duty cycle
     for a simple heater/cooler
  if ( error > 0){
    integral = next_integral;
    }

  }

  else{
    duty = pid_out/3.3;
    integral = next_integral;
  }
  duty = (pow(2, PWM_res) - 1)*(duty);

  // std::string sending = fmt::format("{0}, {1}, {2},
     {3}, {4}", P, I, D, error, duty);
  // Serial.println(String(sending.c_str()));
  // Serial.flush();
}
//Otherwise turn off control signals.
```

```
      else
      {
        ledcWrite(0, 0);
        ledcWrite(1, 0);
      }

      //Check how long everything took in microseconds.
      int elapsed = timerReadMicros(timer);

      if(elapsed <= sampling_period)
      {
        delayMicroseconds(sampling_period - elapsed);
        message = 100; //Reset message flag to show nothing.
      }
      else
      {
        delayMicroseconds(sampling_period);
        message = 0; //Set message flag for sampling too fast.
      }


    //Otherwise deactivate PWM outputs.
    }
    else
    {
      ledcWrite(0, 0);
      ledcWrite(1, 0);
    }
}
```

List of Tables

1 P Controller Tuned Parameters 5
2 P Controller Step Response Metrics (2% Settling Criterion) 5
3 PI Controller Tuned Parameters 8
4 PI Controller Step Response Metrics (2% Settling Criterion) 8
5 PD Controller Step Response Metrics 11
6 Table comparing closed loop PID controller tuned for reference tracking against the open loop controller 14

List of Figures

1 Accuracy comparison of G_{11}'s transfer function 3
2 Simulink model used for reference tracking with no disturbance (Figure 1 from the practical guide). The controller $C(s)$ drives the plant G_{11} in a unity negative feedback loop. 4
3 Simulink model used for reference tracking with disturbance (Figure 2 from the practical guide). The PWM2 input passes through G_{12} and is added to the plant output, simulating the thermal disturbance from transistor 2. 5
4 Closed-loop vs open-loop step response of the P controller. The closed-loop response reaches the reference significantly faster but exhibits overshoot and a steady-state error due to the absence of an integral term. 6
5 Closed-loop response of the P controller with no disturbance. Top: reference trajectory and controlled variable (temperature). Bottom: manipulated variable (PWM1 voltage). 6
6 Closed-loop response of the P controller with disturbance via G_{12}. Top: reference and controlled variable. Bottom: manipulated variable (PWM1) and disturbance input (PWM2). 7
7 Closed-loop vs open-loop step response of the PI controller. The integral term eliminates the steady-state error present in the P controller, with the closed-loop output converging exactly to the 50 °C reference. 9
8 Closed-loop response of the PI controller with no disturbance. Top: reference trajectory and controlled variable (temperature). Bottom: manipulated variable (PWM1 voltage). 9
9 Closed-loop response of the PI controller with disturbance via G_{12}. Top: reference and controlled variable. Bottom: manipulated variable (PWM1) and disturbance input (PWM2). 10
10 Simulink Model used for PD Controller 11
11 Comparing the closed loop PD controller step response to the open loop step response. 12
12 PD Control with no disturbance 12
13 PD Control with disturbance 13
14 Comparing the closed loop step response for the PID reference tracking controller to the open loop step response 14
15 PID reference tracking control with no disturbance 15
16 PID reference tracking control with disturbance 15

17 Simulink Model used for PID Disturbance Rejection Controller 16
18 Comparing the closed loop step response of the PID disturbance rejection controller to the open loop step response 17
19 Simulink closed-loop step response for the disturbance-rejection PID controller. The bottom figure demonstrates the controller compensating for the external heat disturbance caused by transistor 2. 18
20 Software execution flow of the discrete PID algorithm, highlighting first-run derivative protection and integral anti-windup logic. 20
21 Measured Results for PID reference tracking for state 1 21
22 Measured Results for PID reference tracking for state 2 21
23 Measured Results for PID reference tracking for state 3 22
24 Measured Results for PID reference tracking for state 4 22
25 Measured Results for PID disturbance rejection tracking for state 1 . 23
26 Measured Results for PID disturbance rejection tracking for state 2 . 23
27 Measured Results for PID disturbance rejection tracking for state 3 . 24
28 Measured Results for PID disturbance rejection tracking for state 4 . 24